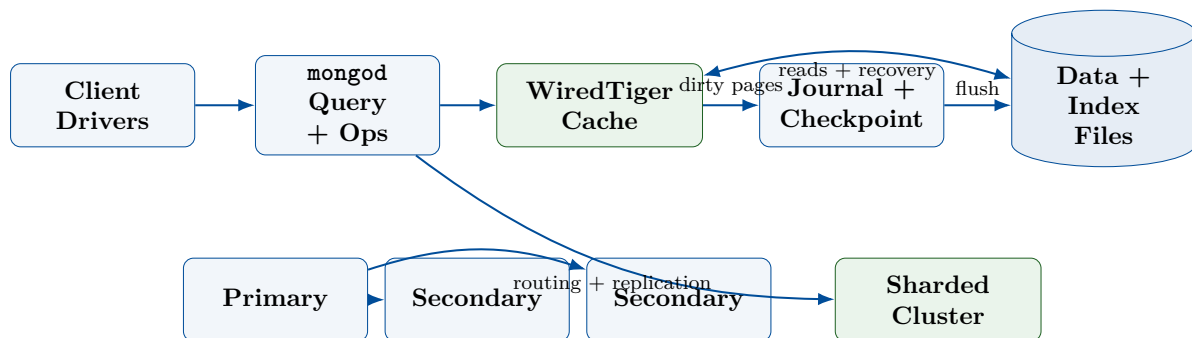

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

MongoDB Master Data Engineer and Senior Architect

Architecture, Aggregation, Security, and DataOps for production MongoDB systems



Inside this book



PhD Aned Esquerra Arguelles

6 milestones, 24 weeks, production architecture, and hands-on labs

July 31, 2026

Preface

Welcome to *MongoDB Master Data Engineer and Senior Architect*. This book is a rigorous, hands-on academic guide for engineers who need to design, operate, and defend MongoDB platforms under production constraints. It moves from core storage-engine mechanics to aggregation, sharding, security, Atlas, vector search, Kubernetes, and DataOps.

The book is written as the theory-and-architecture companion to a 24-week MongoDB training program. Where labs ask you to configure clusters and ship data products, this book explains why each decision matters: which failure mode it prevents, which latency or durability trade-off it introduces, and how a senior architect evaluates the design.

Who This Book Is For

This book is for data engineers, platform engineers, database administrators, cloud architects, and senior technical candidates preparing for MongoDB-heavy architecture interviews. It assumes basic database knowledge, comfort with shell commands, and willingness to read execution metrics. No prior storage-engine expertise is required; Chapter 1 builds the vocabulary for cache, checkpoints, journals, and recovery.

How This Book Is Organized

The book is organized as **6 Parts = 6 Milestones**, with **24 chapters = 24 weeks** plus appendices:

- **Part I** covers WiredTiger internals, schema patterns, migration, and validation.
- **Part II** covers indexing, query plans, and the aggregation framework.
- **Part III** covers replica sets, sharding, distributed transactions, and global resilience.
- **Part IV** covers performance tuning, change streams, observability, and incident response.
- **Part V** covers authentication, encryption, Atlas operations, and vector search.
- **Part VI** covers database change management, Kubernetes, disaster recovery, and the production capstone.

Each chapter opens with **Learning Objectives** and closes with **Further Reading**, **Key Takeaways**, and **Exercises**. Hands-on sections use `mongosh`, JavaScript, YAML, and PowerShell-friendly command blocks.

Reading Paths

- **New to MongoDB architecture:** read Chapters 1–4 in order before jumping to indexes or sharding.
- **Data engineer building pipelines:** read Parts I, II, and IV, then focus on change streams and DataOps.
- **Senior architect or DBA:** read Chapters 1, 5, 9–14, 17–19, and 21–24 for design-review depth.
- **AI and search practitioner:** read the Atlas and vector-search chapters after mastering indexing and aggregation.
- **Interview preparation:** work every chapter’s conceptual and hands-on exercises, especially the architecture scenarios.

Conventions

Code identifiers and file names appear in **monospace**. Listings use the shared style defined in the preamble; MongoDB shell examples are JavaScript listings, and configuration files are YAML listings. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations refer to the bibliography at the back of the book. Treat every metric, write concern, shard key, index, and role assignment as an architectural decision with an operational cost.

Contents

0	Getting Started: MongoDB Atlas, Local Deployment, and Tooling	1
0.1	Why Chapter 0 Exists	1
0.2	Two Complementary Setup Paths	2
0.2.1	Path A: Atlas for Managed Cloud Practice	2
0.2.2	Path B: Local Community Server for Engine Labs	2
0.3	Creating a Free MongoDB Atlas Environment	2
0.3.1	Sign Up Without a Credit Card	2
0.3.2	Deploy an M0 Shared Cluster	3
0.3.3	Understand M0 Limits	4
0.4	Securing Atlas Before Connecting	4
0.4.1	Create a SCRAM Database User	4
0.4.2	Configure Network Access	4
0.4.3	Retrieve the Connection String	5
0.5	Installing Local MongoDB Community Server on Windows	5
0.5.1	Choose the MSI Installer and Service Option	5
0.5.2	Inspect and Manage the Windows Service	5
0.5.3	Understand <code>mongod.cfg</code>	6
0.5.4	When to Use Local Instead of Atlas	6
0.6	Installing Client and Data Tools	7
0.6.1	Install <code>mongosh</code>	7
0.6.2	Install MongoDB Compass	7
0.6.3	Install MongoDB Database Tools	7
0.6.4	Install Python and PyMongo	7
0.7	Connecting from Shell, Compass, and Python	8
0.7.1	Connect to Atlas with <code>mongosh</code>	8
0.7.2	Connect to Localhost with <code>mongosh</code>	8
0.7.3	Connect with PyMongo	9
0.8	Cost and Quota Guardrails	9
0.8.1	Keep the Free Tier Free	9
0.8.2	Pause, Terminate, or Scale Down Deliberately	10
0.8.3	Set Billing Alerts Before Scaling Up	10
0.9	Cross-Cloud Setup Equivalence	10
0.10	Verification Checklist	11
0.10.1	Atlas Verification	11
0.10.2	Local Verification	11
0.11	Further Reading	12
0.12	Exercises	12

I	WiredTiger Internals, Schema Patterns, and Migration	14
1	MongoDB Core Architecture and the WiredTiger Storage Engine	15
1.1	Week 1 Roadmap: From Process to Persistent Bytes	16
1.2	Monday: MongoDB Process Architecture	17
1.2.1	The <code>mongod</code> Daemon as the Database Boundary	17
1.2.2	Connection Handling, Threading, and Pools	17
1.2.3	Memory Ownership: Process RSS, WiredTiger Cache, and OS Page Cache	18
1.2.4	BSON Documents and the Storage-Engine API	18
1.3	Tuesday: WiredTiger Internals	18
1.3.1	B+Tree Data and Index Files	18
1.3.2	Pages, Allocation, and Reconciliation	19
1.3.3	The WiredTiger Cache and Eviction Policy	19
1.3.4	Configurable Cache Size	20
1.3.5	Compression: Snappy, Zstandard, and Prefix Compression	20
1.4	Wednesday: The Write Path, Journaling, Checkpointing, and Recovery	20
1.4.1	From Client Write to Dirty Page	20
1.4.2	Write-Ahead Logging and the Journal	21
1.4.3	Checkpointing and the 60-Second Rhythm	21
1.4.4	Crash Recovery and Durability Guarantees	21
1.5	Thursday Hands-On Project: Configure and Observe WiredTiger	22
1.5.1	Local Configuration	22
1.5.2	PowerShell and Shell Commands	23
1.5.3	Indexes and Workload Driver	23
1.5.4	Metrics to Read	24
1.5.5	Interpreting the Experiment	24
1.6	Friday Use Case and Architecture: Financial Ledger Engine	24
1.6.1	Scenario and Requirements	24
1.6.2	Document and Index Shape	25
1.6.3	Cache Allocation Versus OS Page Cache	25
1.6.4	Architecture Diagram	26
1.6.5	Operational Guardrails	26
1.7	Interview Q&A	26
1.7.1	Concepts and Trade-offs	26
1.8	Further Reading	27
1.9	Exercises	27
1.10	Capstone Project: A WiredTiger Cache-Tuning Lab for a Financial Ledger	28
2	Advanced Schema Design Patterns	35
2.1	Week 2 Roadmap: Designing Documents Around Workloads	36
2.2	Monday: Core Design Patterns	37
2.2.1	Polymorphic Pattern	37
2.2.2	Attribute Pattern	37
2.2.3	Bucket Pattern	38
2.2.4	Outlier Pattern	38
2.3	Tuesday: Operational Patterns	39
2.3.1	Computed Pattern	39
2.3.2	Subset Pattern	39
2.3.3	Extended Reference Pattern	40
2.3.4	Schema Versioning Pattern	40
2.4	Wednesday: Data Lifecycle Patterns	41
2.4.1	Time-To-Live Pattern	41

2.4.2	Pre-allocation Pattern	41
2.4.3	Archival Pattern	42
2.5	Thursday Hands-On Project: Refactor a Product Catalog	42
2.5.1	Scenario	42
2.6	Friday Use Case and Architecture: IoT Metrics at 100,000 Events per Second	44
2.6.1	Scenario and Requirements	44
2.6.2	Bucketed Document Shape	45
2.7	Interview Q&A	46
2.7.1	Concepts and Trade-offs	46
2.8	Further Reading	46
2.9	Exercises	47
2.10	Capstone Project: Design and Validate a Bucket-Pattern IoT Metrics Store	47
3	Modeling Relationships and Modern Anti-Patterns	53
3.1	Week 3 Roadmap: Relationships, Limits, and Validation	54
3.2	Monday: Embedding versus Referencing	55
3.2.1	Decision Criteria	55
3.2.2	1:1 Relationships	55
3.2.3	1:N Few Relationships	56
3.2.4	1:N Many Relationships	56
3.2.5	N:M Relationships	56
3.3	Tuesday: Document Anti-Patterns	56
3.3.1	Unbounded Arrays	56
3.3.2	Bloated Documents and the 16 MB Limit	57
3.3.3	Massive Collections Without Shape Strategy	57
3.4	Wednesday: JSON Schema Validation	58
3.4.1	Validation Goals	58
3.4.2	Structural Constraints, Types, and Enums	58
3.5	Thursday Hands-On Project: Enterprise HR Validator	58
3.5.1	Scenario	58
3.6	Friday Use Case and Architecture: Remediating 16 MB Activity-Log Failures	61
3.6.1	Scenario and Root Cause	61
3.7	Interview Q&A	61
3.7.1	Concepts and Trade-offs	61
3.8	Further Reading	62
3.9	Exercises	63
3.10	Capstone Project: Remediate a 16 MB Activity-Log Failure	63
4	Relational to Document Migration Strategy	69
4.1	Week 4 Roadmap: Migration as Document-Model Modernization	70
4.2	Monday: Analyzing RDBMS Schemas	71
4.2.1	From Tables to Aggregate Boundaries	71
4.2.2	Finding 3NF Tables to Merge	71
4.2.3	Merge, Reference, or Duplicate	72
4.3	Tuesday: MongoDB Relational Migrator	72
4.3.1	Mapping Model and Transformation Rules	72
4.3.2	CDC Setup	72
4.3.3	Type and Constraint Translation	72
4.4	Wednesday: Zero-Downtime Migration Patterns	73
4.4.1	Dual-Writes and Shadow Reads	73
4.4.2	CDC Replication Pipelines	73
4.4.3	Rollback and Data-Loss Boundaries	74

4.5	Thursday Hands-On Project: E-Commerce Relational Migrator Lab	74
4.5.1	Scenario	74
4.6	Friday Use Case: Zero-Downtime 15 TB PostgreSQL Migration	77
4.6.1	Scenario and Requirements	77
4.6.2	Operational Runbook	77
4.7	Interview Q&A	78
4.7.1	Migration Design and Operations	78
4.8	Further Reading	78
4.9	Exercises	79
4.10	Milestone 1 Capstone: Relational Monolith Modernization Engine	79
A	Cross-Cloud Service Equivalence	84
A.1	Object and Data-Lake Storage	84
A.2	Managed Relational Databases	85
A.3	Elastic Compute and Analytical Warehouses	85
A.4	Data Organization and the Medallion Pattern	85
A.5	Document Data Modeling	86
A.6	Globally Distributed and NoSQL Databases	86
A.7	Managed Apache Spark	87
A.8	Query Acceleration and Materialized Views	87
A.9	In-Memory Caching and Hybrid Ingestion	87
A.10	Wide-Column and Key-Value NoSQL	88
A.11	Data Protection: Cloning, Time Travel, and Migration	88
	Glossary	89

List of Figures

1.1	MongoDB write path through <code>mongod</code> , WiredTiger cache, journal, checkpoint, and durable data files.	22
1.2	Financial ledger architecture balancing journal durability, WiredTiger cache, checkpoints, replica-set replication, and downstream change streams.	26
1.3	Capstone lab architecture for balancing WiredTiger cache, journal write-ahead logging, checkpoint flushing, durable files, and the operating-system page cache.	29
2.1	Bucket Pattern grouping many time-series readings into one bounded document with shared metadata and summary fields.	38
2.2	Subset Pattern keeping a small hot subset near the parent document while preserving full history in a separate collection.	40
2.3	Bucketed IoT metrics architecture reducing document count and repeated metadata for 100,000 events per second.	45
2.4	Capstone architecture for validating bucketed IoT metrics, hot MongoDB retention, rollups, and archival flow.	49
3.1	Embedding versus referencing decision tree for bounded relationships, shared lifecycle, and dominant read locality.	55
3.2	Unbounded arrays turn a convenient embedded design into a size-limit and write-amplification failure.	57
3.3	Parent-referenced activity events and capped recent diagnostics replace an unbounded embedded activity array.	61
3.4	Capstone reference architecture with bounded user profiles, parent-referenced events, capped diagnostics, and audit archival.	64
4.1	Normalized tables collapse into bounded documents, while unbounded event history remains referenced.	71
4.2	Dual-write and CDC cutover pipeline with shadow-read comparison and rollback routing.	73
4.3	Zero-downtime 15 TB migration architecture with CDC, reconciliation, canary reads, durable write log, and rollback routing.	77
4.4	Milestone 1 capstone architecture from source RDBMS through CDC into validated Atlas documents and benchmark evidence.	80

List of Tables

1.1	Example before-and-after comparison for the cache-tuning recommendation. . . .	33
-----	--	----

Getting Started: MongoDB Atlas, Local Deployment, and Tooling

Learning Objectives

- Create a MongoDB Atlas account, organization, project, and forever-free M0 shared cluster without supplying a credit card.
- Secure Atlas before the first connection by creating a SCRAM database user, adding a narrow IP access-list entry, and retrieving a `mongodb+srv` connection string.
- Install MongoDB Community Server on Windows as a service and locate the default `dbPath`, log path, and `mongod.cfg` configuration file used by the local WiredTiger labs.
- Install and verify `mongosh`, MongoDB Compass, MongoDB Database Tools, and the PyMongo Python driver in a Windows and PowerShell environment.
- Connect to both Atlas and `localhost`, run a health check, insert and query a document, and confirm the active storage engine with `db.serverStatus()`.
- Apply cost and quota guardrails so free-tier learning environments do not become accidental paid deployments.

0.1 Why Chapter 0 Exists

Chapter 1 begins inside the MongoDB server, following documents through the `mongod` process and the WiredTiger storage engine. Before that analysis is useful, every reader needs two working environments. The first is **MongoDB Atlas**, MongoDB’s managed cloud database platform. Atlas is the easiest place to practice database users, network access, connection strings, Compass, drivers, and cloud-style operational controls. The second is a **local MongoDB Community Server** on Windows. Local deployment is necessary for storage-engine labs because it exposes the server configuration file, service lifecycle, local logs, data directory, and WiredTiger files that managed Atlas intentionally hides from customers.

These two paths are complementary rather than competing. Atlas teaches managed service thinking: project boundaries, access lists, credentials, cluster tiers, monitoring, and cost controls. Local Community Server teaches node-level thinking: `mongod.cfg`, `dbPath`, Windows services, log files, local disks, and storage-engine behavior. Senior data engineers need both. A managed platform removes toil, but it does not remove the need to understand the database engine’s behavior; a local server reveals internals, but it does not replace managed production governance.

Key Terms

Atlas: MongoDB’s managed cloud database platform. **M0:** Atlas’s forever-free shared cluster tier, limited to 512 MB of storage and shared RAM. **SCRAM:** Salted Challenge Response Authentication Mechanism, the username/password authentication mechanism used for MongoDB database users. **IP access list:** Atlas network rule that permits connections only from configured source IP addresses or CIDR ranges. **SRV URI:** A `mongodb+srv://` connection string that uses DNS SRV records to discover cluster hosts.

0.2 Two Complementary Setup Paths

0.2.1 Path A: Atlas for Managed Cloud Practice

Atlas is the fastest path to a real MongoDB deployment. It provides a web console, managed backups on paid tiers, monitoring, cloud-provider placement, connection helpers, database-user management, and network access controls. The free M0 shared cluster is appropriate for this book’s introductory driver, shell, Compass, and CRUD exercises. It is not appropriate for high-volume performance testing, production workloads, or experiments that require direct access to the host operating system.

For Chapter 0, Atlas provides three capabilities that matter immediately. First, it gives you a stable remote MongoDB endpoint that looks like production from a client perspective. Second, it requires explicit security setup before a connection is allowed, which builds the correct habit early. Third, it makes cost and quota choices visible through cluster tiers, project settings, and billing controls.

0.2.2 Path B: Local Community Server for Engine Labs

MongoDB Community Server on Windows runs a local `mongod.exe` process, usually installed as a Windows service. It stores database files under the default data directory and writes logs to a local log file. This path is used throughout the WiredTiger chapters because students can inspect service state, stop and start the server, read configuration, change lab-only settings, and observe how local storage responds.

A local server is not automatically safer than Atlas. It can still store sensitive data, consume disk, bind to network interfaces, and run with overly broad permissions. For this book, keep the local server bound to `localhost`, use synthetic lab data, and avoid importing customer data or secrets.

Definition

Use **Atlas** when the exercise is about managed connectivity, credentials, Compass, drivers, cloud monitoring, or operational guardrails. Use **local Community Server** when the exercise is about the Windows service, `mongod.cfg`, local files, WiredTiger storage behavior, logs, or controlled restarts.

0.3 Creating a Free MongoDB Atlas Environment

0.3.1 Sign Up Without a Credit Card

Start at the MongoDB Atlas sign-up page and create an account with an email address, Google account, or GitHub account. The exact screen order can change, but the required decisions are stable. Choose the free path, verify your email if prompted, and continue into the Atlas console.

Atlas does not require a credit card to create an M0 free cluster. If a screen offers a paid plan, trial upgrade, or sales-assisted path, choose the free shared-cluster option instead.

After sign-up, Atlas organizes resources in two levels. An **Organization** is the top-level administrative boundary. A **Project** is the operational boundary that contains clusters, database users, network access rules, alerts, and integrations. For this book, use one organization and one project named something recognizable, such as **MongoDB Book Labs**. Keep names boring and descriptive; the name will appear in connection helpers and console navigation.

1. Open Atlas and sign in.
2. Create or select an Organization.
3. Create a Project for this book's labs.
4. Confirm that you are in the intended Project before creating any cluster.

Note

Atlas documentation and console labels evolve, but the security model remains consistent: a project owns clusters, database users, IP access-list entries, and connection helpers. If a console page does not match this chapter exactly, search within the Atlas UI for **Database**, **Database Access**, and **Network Access**.

0.3.2 Deploy an M0 Shared Cluster

Within the project, choose to build or deploy a database. Select the **M0 Free** shared tier. The M0 tier is free forever for qualifying accounts and projects, but it has important limits: it provides 512 MB of storage and shared RAM on shared infrastructure. This is enough for command-line practice, small sample datasets, Compass exploration, and driver tests. It is not a performance benchmark environment.

Atlas asks for a cloud provider and region. Choose the provider and region nearest to you or nearest to the environment where your applications will run. For this book, the choice does not affect the concepts. What matters is that you can identify where the cluster lives and that you avoid accidentally selecting a paid dedicated tier. The console may recommend AWS, Azure, or Google Cloud regions depending on availability.

1. Select **M0 Free** or **Free Shared** as the cluster tier.
2. Choose a cloud provider such as AWS, Azure, or Google Cloud.
3. Choose a nearby region that supports M0.
4. Keep optional paid features disabled unless explicitly required by a later lab.
5. Give the cluster a simple name such as **Cluster0** or **book-m0**.
6. Create the cluster and wait for provisioning to complete.

Common Pitfall

Accidentally choosing a paid tier. The Atlas cluster creation screen can show multiple tiers near each other. Verify that the selected tier is M0, the price shows free, and no credit-card step is required. If the page asks you to configure cluster size, disk scaling, backup policy, or dedicated hardware, re-check the tier before continuing.

0.3.3 Understand M0 Limits

The M0 tier is intentionally small. Its most important constraints for this book are 512 MB of storage and shared RAM. Shared RAM means the environment is suitable for functional learning, not reliable performance measurement. Some Atlas features are unavailable or limited on M0 compared with paid tiers. Treat M0 as a notebook-sized lab environment: excellent for learning connection behavior, CRUD syntax, simple indexes, basic aggregation, and driver setup; inappropriate for sustained load, production data, sensitive workloads, or engine-level experiments.

If a later exercise asks you to reason about cache pressure, checkpoint I/O, local storage files, or service restart behavior, use the local Community Server path. Atlas M0 hides those details because MongoDB operates the underlying hosts.

0.4 Securing Atlas Before Connecting

Atlas requires two independent security decisions before a client can connect: **who** is allowed to authenticate and **where** connections are allowed to originate. The database user answers the first question. The IP access list answers the second.

0.4.1 Create a SCRAM Database User

Open **Database Access** in the Atlas project and create a database user. Choose password authentication, which uses MongoDB's SCRAM mechanism. Use a username dedicated to these labs, such as **bookUser**. Generate a strong password and store it in your password manager. Do not reuse your Atlas console password, operating-system password, or source-control password.

For privileges, choose the smallest role that still supports the lab. Early chapters need read and write access to a lab database. A practical choice is **readWrite** on a database named **booklabs**. Avoid **Atlas Admin** and broad administrative roles for routine work. You can create separate users later for monitoring, backup, or application tests.

Definition

SCRAM authentication stores salted verifier material rather than a plaintext password. Clients prove knowledge of the password through a challenge-response exchange. This is the standard username/password path for MongoDB database users.

0.4.2 Configure Network Access

Open **Network Access** and add an IP access-list entry. For a personal workstation lab, use **Add Current IP Address**. Atlas detects your public source IP and creates a rule for that address, usually as a single-address CIDR entry. Add a short comment such as **home workstation** or **training laptop**. If your ISP changes your address, you may need to update this entry later.

Corporate VPNs, hotel networks, mobile hotspots, and cloud-hosted development environments may change the apparent source IP. If a connection worked yesterday and fails today with a network timeout, check the current public IP and compare it with the Atlas Network Access page before changing code.

Common Pitfall

Avoid 0.0.0.0/0 for normal labs. This CIDR range allows connections from the entire IPv4 internet. A database user and password are still required, but exposing the cluster broadly increases attack surface and makes password hygiene more critical. Use a narrow current-IP rule for this book unless a controlled corporate or cloud environment requires a documented CIDR range.

0.4.3 Retrieve the Connection String

After the cluster is running and security is configured, open the cluster's **Connect** workflow. Choose **Shell**, **Compass**, or **Drivers**; each option shows a connection string. For modern Atlas clusters, the string usually starts with `mongodb+srv://`. It contains a username placeholder, a cluster DNS name, options such as `retryWrites=true`, and an application name.

A typical Atlas shell URI looks like this, with placeholders for credentials and cluster name:

```
1 mongodb+srv://bookUser:<password>@cluster0.example.mongodb.net/booklabs?
  retryWrites=true&w=majority&appName=BookLabs
```

Listing 1: Atlas SRV URI pattern; replace placeholders before use.

In PowerShell, wrap the URI in double quotes when passing it to `mongosh`. If your password contains reserved URI characters such as `@`, `:`, `/`, `?`, or `#`, either let the Atlas helper generate the full command or URL-encode the password. Do not paste a real password into notes, screenshots, source files, or shared chat messages.

0.5 Installing Local MongoDB Community Server on Windows**0.5.1 Choose the MSI Installer and Service Option**

Download MongoDB Community Server for Windows from MongoDB's official download page. Choose the current stable Community Server version that matches this book's timeframe unless a class environment specifies a version. Use the MSI installer. During installation, choose **Complete** unless you need a custom path, and keep the option to install MongoDB as a Windows service enabled.

The service installation creates a managed `mongod.exe` process that Windows can start automatically. It also creates a configuration file and default directories. The common defaults on Windows are:

- Program files: `C:\Program Files\MongoDB\Server\<version>\bin`
- Configuration file: `C:\Program Files\MongoDB\Server\<version>\bin\mongod.cfg`
- Data path: `C:\Program Files\MongoDB\Server\<version>\data`
- Log path: `C:\Program Files\MongoDB\Server\<version>\log\mongod.log`

Some installer versions and older manual-install guides use `C:\data\db` as the data directory. Trust the installed `mongod.cfg` for your machine. This book's commands assume the service is installed and bound to localhost.

0.5.2 Inspect and Manage the Windows Service

Open PowerShell as a normal user for read-only checks. Use an elevated PowerShell session only when starting, stopping, or changing service configuration requires administrator rights. The service is usually named `MongoDB`.

```

1 Get-Service -Name MongoDB
2 Start-Service -Name MongoDB
3 Get-Service -Name MongoDB

```

Listing 2: Checking the local MongoDB Windows service with PowerShell.

Expected service output includes a status such as **Running**. If the service fails to start, read the log path listed in `mongod.cfg`; configuration mistakes, missing directories, and permission problems usually appear there before they appear in client tools.

0.5.3 Understand `mongod.cfg`

The Windows MSI installer writes a YAML configuration file. Your exact version path may differ, but the shape is similar to Listing 3. The most important early settings are `storage.dbPath`, `systemLog.path`, `net.bindIp`, and `net.port`. Keep `bindIp` on `127.0.0.1` for local labs unless a later chapter explicitly discusses secured remote access.

```

1 # mongod.cfg
2 storage:
3   dbPath: C:\Program Files\MongoDB\Server\8.0\data
4
5 systemLog:
6   destination: file
7   logAppend: true
8   path: C:\Program Files\MongoDB\Server\8.0\log\mongod.log
9
10 net:
11   port: 27017
12   bindIp: 127.0.0.1
13
14 processManagement:
15   windowsService:
16     serviceName: MongoDB
17     displayName: MongoDB Server
18     description: MongoDB Server

```

Listing 3: Representative Windows `mongod.cfg` for local labs.

Note

MongoDB configuration is YAML, so indentation matters. Use spaces, not tabs. If `mongod` rejects the file, check indentation, path spelling, and whether Windows backslashes were copied exactly.

0.5.4 When to Use Local Instead of Atlas

Use local Community Server when you need to restart the server repeatedly, inspect logs without a web console, alter lab-only server settings, observe data files, or test `db.serverStatus().storageEngine` against a known host. Use Atlas when you need managed connectivity from Compass or a driver, shared cloud access for class, or realistic project-level security controls.

Do not use local Community Server as a production substitute merely because it is easy to install. Production systems need backups, monitoring, patching, least-privilege access, TLS, authentication, auditing where required, tested recovery, and usually replica sets. Atlas provides many of those operational layers as managed services; a local single-node server does not.

0.6 Installing Client and Data Tools

0.6.1 Install mongosh

mongosh is the modern MongoDB Shell. It is the primary command-line tool for this book. Install it from MongoDB's download page or through an approved package manager if your environment provides one. On Windows, verify that **mongosh.exe** is on your **PATH**; otherwise, open it from its installation directory or add the bin directory to the user **PATH**.

```
1 mongosh --version
2 mongosh "mongodb://localhost:27017/booklabs"
```

Listing 4: Verifying mongosh from PowerShell.

0.6.2 Install MongoDB Compass

MongoDB Compass is the graphical desktop client. It is useful for inspecting collections, sampling documents, building simple aggregations, viewing indexes, and confirming that shell or driver changes created the expected data. Install Compass from MongoDB's official download page. Use the Atlas connection helper to copy a Compass URI, paste it into Compass, and save it with a lab-specific name. Do not save production passwords in shared desktop profiles.

Compass is not a replacement for **mongosh**. The shell is better for repeatable exercises, exact commands, and transcript-based troubleshooting. Compass is better for visual inspection and quick exploration.

0.6.3 Install MongoDB Database Tools

MongoDB Database Tools are separate command-line utilities that include **mongodump**, **mongorestore**, **mongoimport**, and **mongoexport**. Install them from MongoDB's Database Tools download page and add their **bin** directory to **PATH**. They are intentionally separate from **mongosh**; installing the shell does not guarantee that dump, restore, import, and export tools are available.

```
1 mongodump --version
2 mongorestore --version
3 mongoimport --version
```

Listing 5: Verifying MongoDB Database Tools.

This book uses these tools for controlled import and recovery exercises. Keep dumps in a project lab folder, not beside source code unless the repository explicitly allows sample data. Dumps can contain sensitive data, so treat them as database backups.

0.6.4 Install Python and PyMongo

The Python driver, PyMongo, lets Python applications talk to MongoDB. Use a virtual environment for labs so driver versions are isolated from system Python packages. In a Windows PowerShell environment, create the environment inside your working folder, activate it, install PyMongo, and run a short test script.

```
1 py -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 python -m pip install --upgrade pip
4 python -m pip install pymongo
5 python -m pip show pymongo
```

Listing 6: Creating a Python virtual environment and installing PyMongo.

If PowerShell blocks script activation, use the organization's approved execution-policy approach. For a one-user training workstation, a common choice is `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`; use institutional guidance when working on a managed machine.

0.7 Connecting from Shell, Compass, and Python

0.7.1 Connect to Atlas with mongosh

Use the Atlas connection helper to copy the shell command. It should look like Listing 7, with your actual cluster hostname and username. PowerShell quotes are important because the URI contains characters such as `?` and `&` that should be passed to `mongosh` as part of one argument.

```
1 mongosh "mongodb+srv://bookUser:<password>@cluster0.example.mongodb.net/
  booklabs?retryWrites=true&w=majority&appName=BookLabs"
```

Listing 7: Connecting to Atlas from PowerShell with `mongosh`.

After connection, the prompt changes to show the current database context. Select the lab database explicitly before inserting data.

```
1 use booklabs
2 db.runCommand({ ping: 1 })
3 db.chapter0.insertOne({
4   environment: "atlas",
5   chapter: 0,
6   verifiedAt: new Date()
7 })
8 db.chapter0.find({ environment: "atlas" }).pretty()
```

Listing 8: Basic Atlas verification commands in `mongosh`.

Expected output for the ping is a document containing `ok`: 1. The insert result should include `acknowledged: true` and an `insertedId`. The find command should print the document you inserted.

0.7.2 Connect to Localhost with mongosh

For the local Windows service, use the standard localhost URI. The default port is 27017. If the service is running and bound to 127.0.0.1, the shell connects without Atlas network rules.

```
1 mongosh "mongodb://localhost:27017/booklabs"
```

Listing 9: Connecting to local MongoDB Community Server.

Run the same functional test locally, changing the environment label so you can distinguish records.

```
1 use booklabs
2 db.runCommand({ ping: 1 })
3 db.chapter0.insertOne({
4   environment: "local",
5   chapter: 0,
6   verifiedAt: new Date()
7 })
8 db.chapter0.find({ environment: "local" }).pretty()
9 db.serverStatus().storageEngine
```

Listing 10: Local insert, find, and storage-engine check.

On a standard modern local MongoDB Community Server, the storage-engine output should identify `wiredTiger`. Atlas also runs MongoDB with WiredTiger, but Atlas does not expose the host-level files and service controls used in the local labs. The `serverStatus` command is documented by MongoDB and becomes a recurring diagnostic source in later chapters [22].

0.7.3 Connect with PyMongo

Create a file such as `chapter0_pymongo_check.py` in a lab folder, not in the book source directory unless your instructor requests it. For Atlas, replace the URI with your SRV string. For local testing, use `mongodb://localhost:27017`. The snippet in Listing 11 inserts one document and reads it back.

```

1 from datetime import datetime, timezone
2 from pymongo import MongoClient
3
4 uri = "mongodb://localhost:27017"
5 client = MongoClient(uri, serverSelectionTimeoutMS=5000)
6
7 # Force connection now so setup errors fail before the insert.
8 client.admin.command("ping")
9
10 db = client["booklabs"]
11 result = db.chapter0.insert_one({
12     "environment": "python-local",
13     "chapter": 0,
14     "verifiedAt": datetime.now(timezone.utc),
15 })
16
17 found = db.chapter0.find_one({"_id": result.inserted_id})
18 print(found)
19 client.close()

```

Listing 11: Minimal PyMongo insert and find check.

For Atlas, do not hardcode the password in source control. In a later production-oriented chapter, credentials move into environment variables or a secrets manager. For Chapter 0, the immediate habit is simpler: never commit a real URI containing credentials, never paste it into public notes, and rotate the database user’s password if it leaks.

Common Pitfall

Confusing Atlas login with database authentication. Your Atlas console account manages projects and billing. A MongoDB database user authenticates to the database endpoint. They are separate identities with separate passwords and separate permissions.

0.8 Cost and Quota Guardrails

0.8.1 Keep the Free Tier Free

Atlas M0 is free forever but limited. The safest learning posture is to create exactly one M0 cluster for the book, load only small synthetic datasets, and resist upgrade prompts unless a later chapter explicitly requires paid capacity. M0 storage is limited to 512 MB, and RAM is shared. When you hit a limit, the correct Chapter 0 response is not to upgrade immediately; it is to remove unnecessary test data, drop unused collections, or switch to a local lab where appropriate.

Avoid enabling paid features casually. Dedicated clusters, larger storage, advanced backup settings, private networking, and some integrations can create cost. Read the cluster tier and

price summary before clicking create or modify. If a lab ever requires a paid feature, document the expected cost, duration, cleanup step, and owner before enabling it.

0.8.2 Pause, Terminate, or Scale Down Deliberately

Atlas free clusters are intended to remain available, but paid clusters should be paused or terminated when not needed. Pausing preserves configuration while stopping compute charges where the tier supports it. Terminating deletes the cluster and its data after confirmation. For training projects, prefer disposable data and scripted reloads so cleanup is easy.

Local servers have their own quota risks. A local import can fill a workstation disk. Database dumps can duplicate data unexpectedly. Large log files can grow if a noisy workload repeats. Use small datasets and clean up after each chapter’s exercises.

0.8.3 Set Billing Alerts Before Scaling Up

If you ever add a payment method or move beyond M0, configure Atlas billing alerts immediately. Alerts should notify you before monthly spend exceeds the intended lab budget. In an enterprise environment, connect the project to the organization’s normal cost-management process rather than relying on one student’s email inbox.

Tip

Measure it. Before scaling an Atlas cluster or changing a local server setting, record the current dataset size, operation latency, storage metrics, and reason for the change. After the change, measure again. If the metric did not improve, scale back down and fix the workload instead of keeping unnecessary capacity.

0.9 Cross-Cloud Setup Equivalence

Cross-Cloud Equivalence			
Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days + always free)	Free trial (\$300/90 days + Always Free)
Sign-up guard	Card + root email	Card + Microsoft account	Card + Google account
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
Admin isolation	Never use root daily	Avoid Global Admin daily	Avoid Owner; least privilege
CLI	AWS CLI v2 (<code>aws configure</code>)	Azure CLI (<code>az login</code>)	gcloud CLI (<code>gcloud init</code>)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts
Console	AWS Management Console	Azure Portal	Google Cloud Console
THIS book’s platform, MongoDB, is used two ways — a forever-free Atlas M0 cluster (no card) secured by a database user and an IP allowlist, and a local Community Server for storage-engine labs; by contrast Microsoft Fabric uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card), and Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors.			

0.10 Verification Checklist

A setup is complete only when both paths have been tested. Do not proceed to Chapter 1 with an unverified install. The minimum successful state is: Atlas accepts a shell connection, local MongoDB accepts a shell connection, a test document can be inserted and found in both places, and the local server reports WiredTiger as the storage engine.

0.10.1 Atlas Verification

Run the Atlas connection command copied from the Atlas helper.

```
1 mongosh "mongodb+srv://bookUser:<password>@cluster0.example.mongodb.net/
  booklabs?retryWrites=true&w=majority&appName=BookLabs"
```

Listing 12: Atlas setup verification from PowerShell.

Then run:

```
1 use booklabs
2 db.runCommand({ ping: 1 })
3 db.chapter0.insertOne({ environment: "atlas", ok: true })
4 db.chapter0.findOne({ environment: "atlas" })
```

Listing 13: Expected Atlas verification sequence.

Expected output pattern:

```
1 { ok: 1 }
2 {
3   acknowledged: true,
4   insertedId: ObjectId('...')
5 }
6 {
7   _id: ObjectId('...'),
8   environment: 'atlas',
9   ok: true
10 }
```

Listing 14: Representative successful Atlas output.

If ping fails with an authentication error, re-check the database username, password, and password encoding. If it fails with a timeout, re-check the IP access list and current public IP. If DNS resolution fails for `mongodb+srv`, verify internet access and local DNS settings.

0.10.2 Local Verification

Confirm the Windows service is running, then connect locally.

```
1 Get-Service -Name MongoDB
2 mongosh "mongodb://localhost:27017/booklabs"
```

Listing 15: Local service and shell verification.

Inside `mongosh`, run:

```
1 use booklabs
2 db.runCommand({ ping: 1 })
3 db.chapter0.insertOne({ environment: "local", ok: true })
4 db.chapter0.findOne({ environment: "local" })
5 db.serverStatus().storageEngine
```

Listing 16: Expected local verification sequence.

Representative storage-engine output:

```

1 {
2   name: 'wiredTiger',
3   supportsCommittedReads: true,
4   readOnly: false,
5   persistent: true
6 }
```

Listing 17: Representative WiredTiger storage-engine output.

The exact fields can vary by MongoDB version, but the **name** should be **wiredTiger**. If the local connection fails, check whether the service is running, whether `mongod.cfg` binds to `127.0.0.1`, and whether another process is already using port 27017.

0.11 Further Reading

- MongoDB Manual: Architecture and Components [15].
- MongoDB Manual: WiredTiger Storage Engine [26].
- MongoDB Manual: `serverStatus` command [22].
- MongoDB Atlas documentation for cluster creation, database users, network access, and connection strings.
- MongoDB installation documentation for Windows Community Server, `mongosh`, Compass, and Database Tools.
- PyMongo driver documentation for connection strings, authentication options, and client lifecycle.

Key Takeaways

- Chapter 0 requires two environments: Atlas M0 for managed cloud practice and local Community Server for Windows and WiredTiger labs.
- Atlas connections require both a SCRAM database user and a matching IP access-list entry before `mongosh`, Compass, or PyMongo can connect.
- The Atlas M0 tier is free forever but limited to 512 MB of storage and shared RAM, so it is for learning rather than benchmarking or production.
- The Windows MSI installer normally creates a MongoDB service, a YAML `mongod.cfg`, a local `dbPath`, and a log path; these are essential for Chapter 1's storage-engine work.
- `mongosh`, Compass, Database Tools, and PyMongo solve different jobs and should all be installed before continuing.
- A setup is not complete until `db.runCommand({ ping: 1 })`, insert, find, and `db.serverStatus().storage` have been verified where applicable.

0.12 Exercises

1. **Atlas account and project checklist.** Create an Atlas account, organization, and project for this book. Record only non-secret metadata: organization name, project name, cloud provider, region, and cluster tier. Do not record passwords or full credentialed URIs.

2. **M0 cluster checklist.** Deploy one M0 shared cluster. Confirm that the tier is free, storage is limited to 512 MB, and the cluster shows as available in the Atlas console.
3. **Security checklist.** Create a SCRAM database user with `readWrite` on `booklabs`. Add your current IP address to the Atlas IP access list. Explain in two sentences why `0.0.0.0/0` is not acceptable for ordinary labs.
4. **Tooling checklist.** Install `mongosh`, `Compass`, `Database Tools`, and `PyMongo`. Capture the version output for `mongosh -version`, `mongodump -version`, and `python -m pip show pymongo`.
5. **Atlas verification.** Connect with `mongosh` to Atlas, run `db.runCommand({ ping: 1 })`, insert one document into `booklabs.chapter0`, and find it. Save the transcript with secrets removed.
6. **Local verification.** Install MongoDB Community Server on Windows as a service. Confirm `Get-Service -Name MongoDB` reports `Running`, connect to `mongodb://localhost:27017/booklabs`, insert one document, find it, and run `db.serverStatus().storageEngine`.
7. **Configuration inspection.** Open the local `mongod.cfg` and identify `storage.dbPath`, `systemLog.path`, `net.port`, and `net.bindIp`. Explain why `bindIp: 127.0.0.1` is the correct default for this book's local labs.
8. **Cost guardrail.** Write a one-paragraph cleanup plan for this book's MongoDB resources. Include what you will delete, what you will keep, and what alert or review you would configure before moving beyond M0.

Part I

WiredTiger Internals, Schema Patterns, and Migration

MongoDB Core Architecture and the WiredTiger Storage Engine

Learning Objectives

- Explain the role of the `mongod` daemon, client drivers, connection handling, BSON, and the storage-engine API in MongoDB's process architecture.
- Describe WiredTiger's use of B+Tree files, cache pages, eviction thresholds, compression, and page allocation.
- Trace a write from client acknowledgement through the in-memory cache, write-ahead journal, checkpoint, and crash recovery path.
- Configure `storage.wiredTiger.engineConfig.cacheSizeGB` safely for a local deployment and reason about its interaction with the operating-system page cache.
- Use `db.serverStatus().wiredTiger` metrics to diagnose eviction pressure, dirty-cache behavior, and checkpoint I/O.
- Architect a high-throughput financial ledger deployment that balances durability, throughput, and predictable checkpoint latency.

Cross-Cloud Equivalence

Although this book teaches **MongoDB**'s WiredTiger engine, the sibling cloud tracks share a common storage-foundation layer; the table shows how those object stores align and where MongoDB deliberately differs.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool / Cold	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication (GRS)	Dual/multi-region + Transfer
Online transfer	DataSync	AzCopy	Storage Transfer Service
Offline transfer	Snow Family	Data Box	Transfer Appliance
Design durability	11 nines	11 nines (LRS) / 16 (GRS)	11 nines
Availability SLA	99.9% (Standard)	99.9% (RA-GRS read 99.99%)	99.95% (multi-region)
Encryption at rest	SSE-S3 / SSE-KMS / SSE-C	SSE + CMK via Key Vault	Google-managed / CMEK / CSEK
Access control	IAM & bucket policies, ACLs	Azure RBAC, SAS, POSIX ACLs	IAM, ACLs, signed URLs
Pricing levers	Storage + requests + egress + retrieval	Storage + operations + egress + retrieval	Storage + operations + egress + retrieval
Consistency	Strong read-after-write	Strong read-after-write	Strong (global)

The distinction that matters for this book: object stores and **Snowflake** micro-partitions and **Fabric OneLake** are *remote, immutable* storage layers, whereas **MongoDB**’s WiredTiger is a *node-local, mutable* engine with its own cache, journal, and checkpoints—which is exactly what the rest of this chapter dissects.

1.1 Week 1 Roadmap: From Process to Persistent Bytes

MongoDB is often introduced as a document database, but a senior data engineer must also understand the path from a BSON document in a client driver to bytes persisted in storage files. The Week 1 objective is not to memorize every internal counter. It is to build an operational model: what **mongod** owns, what WiredTiger owns, what the operating system owns, and which metrics prove that the model is healthy. MongoDB’s architecture documentation presents the database as a layered system of clients, server processes, storage, replication, and sharding [15]; this chapter opens the storage layer because later chapters on indexes, sharding, and DataOps depend on it.

Definition

WiredTiger is MongoDB’s default storage engine. It manages the on-disk data and index files, an internal cache, concurrency control, compression, checkpoints, and the journal used to recover committed writes after a crash [26, 33].

A useful mental model is to separate three caches. The **driver and application** may cache connections and objects. **WiredTiger** caches database pages inside the **mongod** process. The **operating system** caches filesystem blocks outside **mongod**. Performance work becomes confusing when these are collapsed into a single word, “memory.”

Key Terms

BSON: MongoDB’s binary document representation. **Dirty page:** a cached page that has changed in memory and must eventually be written to disk. **Checkpoint:** a durable, consistent view of data files at a point in time. **Journal:** write-ahead log used to replay committed changes during recovery. **Eviction:** the act of removing clean or reconciled pages from the WiredTiger cache to make room for active work.

1.2 Monday: MongoDB Process Architecture

1.2.1 The mongod Daemon as the Database Boundary

The `mongod` daemon is the server process that accepts client connections, authenticates requests, parses commands, coordinates query execution, invokes the storage engine, and participates in replication or sharding roles. It is the operational boundary for configuration, logs, process memory, CPU scheduling, file descriptors, and network sockets. A standalone `mongod` may be enough for development, but production MongoDB systems usually run each `mongod` as a member of a replica set or as a shard inside a larger sharded cluster [21, 23].

At the top of the stack, drivers maintain connection pools and send commands over MongoDB’s wire protocol. Inside `mongod`, the command is authorized, parsed, planned, executed, and eventually translated into storage-engine operations. The storage engine is not responsible for user authentication, cluster routing, or the semantics of an aggregation pipeline. It is responsible for storing and retrieving records and indexes under a transactional contract.

Note

Architectural debugging starts by naming the layer. A slow dashboard could be caused by driver pool starvation, a query planner mistake, an index miss, WiredTiger eviction pressure, checkpoint I/O, disk latency, or cross-shard scatter-gather. Each layer has different evidence.

1.2.2 Connection Handling, Threading, and Pools

MongoDB has historically been described as using a thread-per-connection model, but modern deployments include listener, worker, service-entry, and asynchronous execution details that vary by server version and platform. The practical point for data engineers is stable: client concurrency turns into server-side scheduling, memory consumption, locks or tickets, and storage-engine work. Each connection is not free. Drivers should use bounded pools, and server sizing should consider concurrent operations, operation mix, and storage tickets rather than only total connection count.

Thread pools and internal executors help the server avoid unbounded resource growth, but they do not remove backpressure. If writes arrive faster than WiredTiger can reconcile dirty pages, if checkpoints cannot complete smoothly, or if disk latency spikes, the server must delay work somewhere. Senior incident review asks where the queue is visible: driver wait time, operation latency, ticket availability, eviction worker activity, journal commit latency, or checkpoint duration.

Common Pitfall

Treating connection count as throughput. More concurrent clients can reduce throughput if they force cache churn, lock contention, disk queueing, or replication lag.

Start with a bounded driver pool and increase concurrency only when server metrics show unused capacity.

1.2.3 Memory Ownership: Process RSS, WiredTiger Cache, and OS Page Cache

MongoDB process memory includes more than the WiredTiger cache: executable code, thread stacks, connection buffers, aggregation memory, query execution state, indexes being built, replication buffers, and allocator overhead also contribute. WiredTiger's cache is the intentionally managed portion for pages. The operating system page cache is outside the configured WiredTiger cache and can still hold recently read filesystem blocks.

By default, MongoDB chooses a WiredTiger cache size from available memory using rules documented by MongoDB, and administrators can override it with `storage.wiredTiger.engineConfig.cacheSize` [26]. The right value is workload-dependent. Analytical scans and hot indexes benefit from cache, but the OS still needs memory for filesystem cache, networking, compression buffers, and the rest of the host.

Tip

On a dedicated MongoDB host, do not allocate all RAM to WiredTiger. Leave room for the OS page cache and non-cache `mongod` memory. On a shared host, also reserve memory for co-located agents, monitoring, backup tools, and container overhead.

1.2.4 BSON Documents and the Storage-Engine API

MongoDB stores records as BSON documents. BSON preserves document structure, types, arrays, embedded subdocuments, and binary values in a format designed for efficient traversal by the server. The query layer reasons about collections, indexes, predicates, projections, updates, and aggregation stages. The storage-engine API sees lower-level operations: create or open tables, insert records, update records, delete records, scan cursors, maintain index structures, and participate in transactions.

This separation explains why document modeling and storage behavior are connected but not identical. Embedding related data may reduce joins and improve locality, yet large growing arrays can force document moves or create write amplification. Indexes accelerate reads but add write work because every insert or update must maintain each affected index. The architectural skill is to translate a document-model decision into storage-engine consequences.

Definition

The **storage-engine API** is the boundary through which MongoDB's higher-level server asks WiredTiger to persist records and indexes. It lets MongoDB swap storage implementations in principle, while WiredTiger remains responsible for page layout, cache management, compression, logging, and checkpoints.

1.3 Tuesday: WiredTiger Internals

1.3.1 B+Tree Data and Index Files

WiredTiger stores collection data and indexes in B-tree-like structures. B+Trees descend from classic ordered-index research [4]: internal pages guide navigation, leaf pages hold keys and values or references to records, and sorted order enables range scans. In MongoDB, a collection

and each secondary index are separate logical structures backed by WiredTiger files. A point lookup on an indexed field navigates the index tree, retrieves record identifiers, and then fetches the corresponding documents when the plan requires them.

B+Tree organization is why index selectivity and working-set locality matter. A workload that repeatedly touches a small region of an index can stay cache-resident. A workload that jumps randomly across a very large index may turn every query into a page-fault and eviction exercise. Range queries exploit order; unbounded sorts without appropriate indexes force memory pressure elsewhere in the server.

Note

MongoDB indexes are not merely query-planner hints. They are persistent storage structures. Every additional secondary index increases insert, update, checkpoint, backup, and recovery work.

1.3.2 Pages, Allocation, and Reconciliation

WiredTiger operates on pages rather than individual documents in isolation. Pages are read from disk into cache, modified in memory, reconciled into a disk-ready representation, compressed, and written during eviction or checkpoint activity. Page sizes, split behavior, and update chains affect how much work occurs when a hot document or hot key range changes repeatedly.

When a page receives updates, WiredTiger may keep newer versions in memory until reconciliation creates a stable page image. This is a major reason storage engines need cache policy: not all memory is equally easy to evict. Clean pages can be discarded because they already exist on disk. Dirty pages require reconciliation and write I/O before their memory can be reused safely.

1.3.3 The WiredTiger Cache and Eviction Policy

The WiredTiger cache is the working memory for internal pages, leaf pages, update lists, and other storage-engine structures. Two important configuration ideas are **eviction_target** and **eviction_trigger**. The target is the level eviction tries to maintain; the trigger is the level at which application threads may be asked to help evict because pressure is too high. The exact defaults and tunables are version-specific, but the operational pattern is stable: healthy systems evict in the background before foreground operations stall.

Definition

Eviction pressure occurs when WiredTiger cannot keep enough useful pages in cache while also making room for new reads and writes. It appears as high cache usage, many pages evicted by application threads, long reconciliation work, or operations waiting behind storage-engine pressure.

Common Pitfall

Increasing cache size without changing the workload. A larger cache can hide poor indexes or unbounded scans until the data grows again. Tune queries and indexes first, then size cache for the measured working set.

1.3.4 Configurable Cache Size

The `cacheSizeGB` setting is a blunt but important capacity control. A cache that is too small increases reads from disk, evictions, and dirty-page churn. A cache that is too large can starve the OS and increase checkpoint burstiness because more dirty data can accumulate before being flushed. Containers add another complication: the database must be aware of actual cgroup limits and orchestration reservations, not only the physical host memory.

A useful sizing conversation starts with four numbers: total RAM available to the process, non-WiredTiger `mongod` memory, OS and filesystem needs, and the hot data/index working set. For a ledger workload, the hot set may be recent accounts, recent ledger entries, and a small number of compound indexes. For an analytics workload, the hot set may be much wider and less predictable.

Tip

Measure bytes currently in the cache, tracked dirty bytes in the cache, pages read into cache, pages evicted from cache, and application-thread eviction counters before changing `cacheSizeGB`. A setting change without before-and-after metrics is a guess.

1.3.5 Compression: Snappy, Zstandard, and Prefix Compression

WiredTiger supports block compression for collection data. MongoDB commonly uses Snappy by default because it is fast and offers moderate compression. Zstandard can achieve better compression at additional CPU cost. Prefix compression can reduce index storage by storing common key prefixes more compactly, which is valuable for compound keys with repeated leading components.

Compression changes the shape of bottlenecks. Better compression reduces disk footprint and I/O, but it spends CPU during compression and decompression. On a disk-bound workload, Zstandard may improve throughput. On a CPU-bound write-heavy workload, Snappy or less aggressive compression may be better. The senior answer is not “compression on” or “compression off”; it is to compare CPU saturation, cache residency, disk throughput, and checkpoint duration.

1.4 Wednesday: The Write Path, Journaling, Checkpointing, and Recovery

1.4.1 From Client Write to Dirty Page

A write begins at the client driver, which sends an insert, update, delete, or bulk operation to `mongod`. The server validates the command, checks authorization, enters the appropriate transactional context, updates collection data and affected indexes, and hands storage changes to WiredTiger. WiredTiger applies changes in memory first. The modified page becomes dirty because its in-memory state is newer than its durable data-file state.

The acknowledgement returned to the client depends on write concern and journaling semantics. Acknowledging a write does not necessarily mean the checkpoint has already placed that write into the main data files. With journaling enabled, durability can be provided by the journal before the next checkpoint writes the complete page image [27].

1.4.2 Write-Ahead Logging and the Journal

Write-ahead logging is a classic recovery technique. The ARIES recovery paper formalized concepts such as logging changes before data-page persistence and replaying committed work after failure [11]. WiredTiger’s journal plays the same broad architectural role: it records enough information to recover committed changes if the server crashes after acknowledging them but before a checkpoint fully flushes affected data pages.

Definition

Write-ahead logging means the durable log record for a change reaches stable storage before the corresponding data-file page is considered safely persisted. This ordering lets recovery replay committed changes after a crash.

Journaling is sequential compared with scattered data-page writes, which is why it is central to write performance. The data files can be checkpointed in organized batches, while the journal protects recent acknowledged work. On a storage device with volatile write cache or high latency, journal behavior is often the first place to look when acknowledged-write latency spikes.

1.4.3 Checkpointing and the 60-Second Rhythm

MongoDB checkpoints WiredTiger data at regular intervals; a commonly cited default checkpoint interval is approximately 60 seconds, though actual behavior is influenced by workload and version. A checkpoint establishes a consistent durable point in the data files. After a successful checkpoint, older journal records covered by that checkpoint can eventually be removed because the data files themselves contain the changes.

Checkpointing is where many write-heavy systems feel periodic pain. If too much dirty data accumulates, the checkpoint may issue a burst of writes. If disk bandwidth is shared with backups, compaction, analytics, or other services, the checkpoint can compete with foreground traffic. If cache is oversized relative to disk throughput, large dirty working sets can produce longer flush periods.

Common Pitfall

Ignoring checkpoint shape. Average write latency can look acceptable while every 60 seconds a checkpoint creates a tail-latency spike. Watch p95 and p99 latency, not only mean throughput.

1.4.4 Crash Recovery and Durability Guarantees

Crash recovery combines the latest stable checkpoint with the journal records after that checkpoint. On restart, WiredTiger opens the files at a consistent checkpoint state and replays logged operations required to restore acknowledged durable writes. The recovery time depends on how much journal must be processed, storage speed, and the size of metadata and cache warm-up effects.

Write concern and journaling interact with replica-set durability. A write concern of `w:1` can acknowledge after the primary accepts the write according to the configured journal behavior. A write concern of `majority` waits for replication to a majority of voting members under MongoDB’s semantics. Journaling protects against local process or host crash; majority write concern protects against primary loss before secondaries receive the operation. They solve different parts of the durability problem [27, 21].

Note

For financial ledgers, the usual default is to prefer explicit durable acknowledgements and idempotent application-level transaction identifiers. Throughput tuning must not silently weaken the business durability contract.

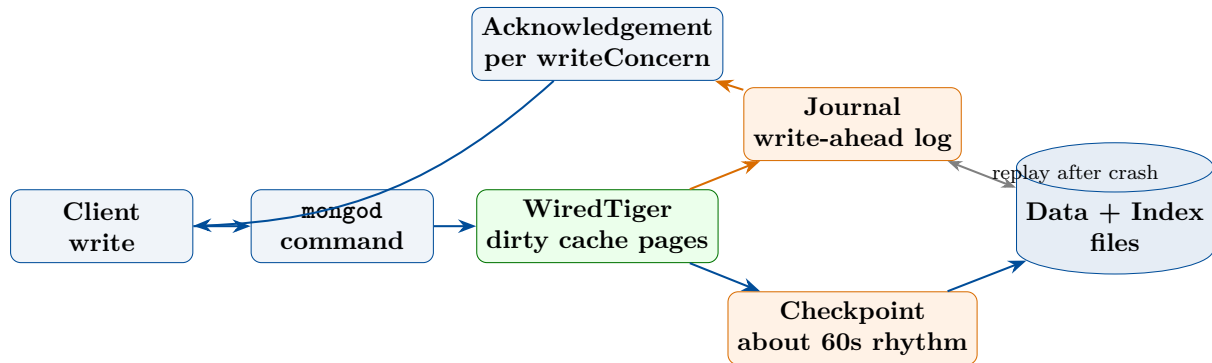


Figure 1.1: MongoDB write path through mongod, WiredTiger cache, journal, checkpoint, and durable data files.

1.5 Thursday Hands-On Project: Configure and Observe WiredTiger

This lab configures a local `mongod`, sets a WiredTiger cache size, loads a large synthetic data set, and inspects eviction metrics. Run it on a development machine, not a production host. The 10-million-document target is intentionally large enough to create pressure; reduce it if your laptop cannot sustain the run.

1.5.1 Local Configuration

Create a dedicated data path and log path. On Windows, run PowerShell as a user with permission to write the selected directories. The following configuration uses a small cache to make eviction visible during the lab.

```

1 storage:
2   dbPath: C:\data\mongodb-wt-lab\db
3   journal:
4     enabled: true
5   wiredTiger:
6     engineConfig:
7       cacheSizeGB: 2
8     collectionConfig:
9       blockCompressor: snappy
10    indexConfig:
11      prefixCompression: true
12 systemLog:
13   destination: file
14   path: C:\data\mongodb-wt-lab\log\mongod.log
15   logAppend: true
16 net:
17   bindIp: 127.0.0.1
18   port: 27017
19 replication:
20   replSetName: wtLabRs
  
```

Listing 1.1: Development-only `mongod.conf` with an explicit WiredTiger cache size.

Tip

For the lab, small cache values reveal eviction behavior faster. For production, size the cache from workload evidence and host constraints, not from a copied lab value.

1.5.2 PowerShell and Shell Commands

The commands below assume MongoDB Database Tools, `mongod`, `mongosh`, Node.js, and `mgeneratejs` are installed and on PATH. They create folders, start the server, initiate a single-node replica set, and load generated records. If you already run a local MongoDB on port 27017, change the port.

```
1 New-Item -ItemType Directory -Force C:\data\mongodb-wt-lab\db
2 New-Item -ItemType Directory -Force C:\data\mongodb-wt-lab\log
3 mongod --config C:\data\mongodb-wt-lab\mongod.conf
4 mongosh --eval "rs.initiate({_id: 'wtLabRs', members: [{_id: 0, host:
  '127.0.0.1:27017'}]})"
5 npm install -g mgeneratejs
6 mgeneratejs '{"_id":"$objectId","accountId":"$guid","postedAt":"$date","amount":
  {"$number":{"min":-5000,"max":5000}},"currency":{"$choose":{"from":["USD",
  "EUR","MXN"]}},"memo":"$lorem"}' -n 10000000 | mongoimport --uri "mongodb
  ://127.0.0.1:27017/ledger" --collection entries --type json
```

Listing 1.2: PowerShell-friendly setup and data generation commands.

The command block is intentionally explicit rather than elegant. In a real benchmark, pin tool versions, isolate the disk, record CPU and storage metrics, and run multiple trials. A single load test is a learning exercise, not a capacity plan.

1.5.3 Indexes and Workload Driver

After loading, create an index that resembles a ledger access pattern: account plus posting time. Then run a mixed insert-and-read workload from `mongosh`. MongoDB shell syntax is JavaScript, so listings use `language=JavaScript`.

```
1 use ledger;
2 db.entries.createIndex({ accountId: 1, postedAt: -1 });
3
4 const sampleAccounts = db.entries.aggregate([
5   { $sample: { size: 1000 } },
6   { $project: { _id: 0, accountId: 1 } }
7 ]).toArray().map(d => d.accountId);
8
9 for (let i = 0; i < 200000; i++) {
10   const accountId = sampleAccounts[i % sampleAccounts.length];
11   db.entries.insertOne({
12     accountId,
13     postedAt: new Date(),
14     amount: NumberDecimal((Math.random() * 1000 - 500).toFixed(2)),
15     currency: "USD",
16     source: "wt-lab",
17     sequence: i
18   }, { writeConcern: { w: "majority", j: true } });
19
20   if (i % 1000 === 0) {
21     db.entries.find({ accountId }).sort({ postedAt: -1 }).limit(20).toArray();
22   }
23 }
24
25 const wt = db.serverStatus().wiredTiger;
26 printjson({
```



```

27 cache: wt.cache,
28 transaction: wt.transaction,
29 log: wt.log,
30 checkpoint: wt.checkpoint
31 });

```

Listing 1.3: mongosh workload and WiredTiger metrics probe.

1.5.4 Metrics to Read

The `serverStatus` command exposes WiredTiger sections for cache, log, transaction, and checkpoint behavior [22]. Field names can vary by version, so treat the categories as more important than a single exact key. Look for the following:

- **Cache occupancy:** bytes currently in cache versus maximum bytes configured.
- **Dirty bytes:** tracked dirty bytes and dirty percentage.
- **Eviction work:** pages evicted by worker threads and by application threads.
- **Read churn:** pages read into cache and pages requested from the cache.
- **Checkpoint behavior:** checkpoint time, bytes written, and recent checkpoint progress.
- **Journal behavior:** log bytes written, sync counts, and latency where exposed.

Note

Application-thread eviction is a useful smoke alarm. It means foreground operations are helping create free cache space instead of simply doing user work. Occasional values are normal; sustained growth during peak traffic deserves investigation.

1.5.5 Interpreting the Experiment

Run the workload three times: once with the 2 GB cache, once with a smaller cache if your host permits it, and once with a larger cache that still leaves OS memory. Compare p95 insert latency, p95 query latency, dirty bytes, pages evicted, and checkpoint duration. If a larger cache lowers read churn but increases checkpoint bursts, the best value may be in the middle.

Common Pitfall

Benchmarking without warming and resetting. The second run can be faster because the OS page cache is warm. Either design the test to include warm-cache behavior intentionally, or restart and document the cache state between runs.

1.6 Friday Use Case and Architecture: Financial Ledger Engine

1.6.1 Scenario and Requirements

A financial ledger is a demanding MongoDB workload because it combines high write volume, strict auditability, predictable reads by account and time, and low tolerance for lost acknowledgements. Assume the platform must ingest payment events, maintain immutable ledger entries, support account-balance queries, and stream changes to risk, fraud, and reporting systems.

The central storage-engine challenge is not only raw throughput. It is avoiding periodic stalls that violate latency objectives. A ledger may sustain 20,000 writes per second for most of a minute and then suffer checkpoint-induced p99 spikes if dirty data accumulates faster than disks

can absorb it. The architecture must keep dirty-cache growth, journal latency, and checkpoint writes within predictable limits.

1.6.2 Document and Index Shape

A ledger entry should be immutable, idempotent, and shaped for the dominant access path. A common pattern is one document per posted ledger event with fields such as `tenantId`, `accountId`, `postedAt`, `transactionId`, `amount`, `currency`, `direction`, and `source`. Compound indexes often begin with `tenant` and `account`, then `time`, because account history is a dominant read. A unique transaction identifier prevents duplicate posting when upstream systems retry.

Indexes should be few and deliberate. Every secondary index adds writes to the write path. Reporting teams may ask for many ad hoc indexes, but a ledger primary store should not become a general exploratory analytics engine. Use change streams, materialized views, or downstream analytical stores for broad reporting when needed.

1.6.3 Cache Allocation Versus OS Page Cache

The ledger’s hot set is usually recent account histories, recent indexes, and uncheckpointed dirty pages. WiredTiger cache should be large enough for hot data and index pages but not so large that checkpoints become bursty or the OS page cache is starved. The OS page cache still helps with filesystem reads, journal files, and data files that WiredTiger has evicted but the operating system can serve without physical I/O.

A reasonable design process is:

1. Estimate hot account and index working set for the peak hour.
2. Reserve memory for non-cache `mongod` usage, connections, aggregation, monitoring, and the OS.
3. Configure WiredTiger cache below the remaining memory rather than at the full host capacity.
4. Set write concern explicitly, usually requiring journaled majority acknowledgement for committed ledger events.
5. Monitor dirty-cache percentage, application-thread eviction, checkpoint duration, journal latency, replication lag, and disk queue depth.

Tip

For high-throughput ledgers, reducing checkpoint spikes often requires both storage and data-model work: fast durable disks, bounded indexes, right-sized cache, batched ingestion, and downstream reporting offload.

1.6.4 Architecture Diagram

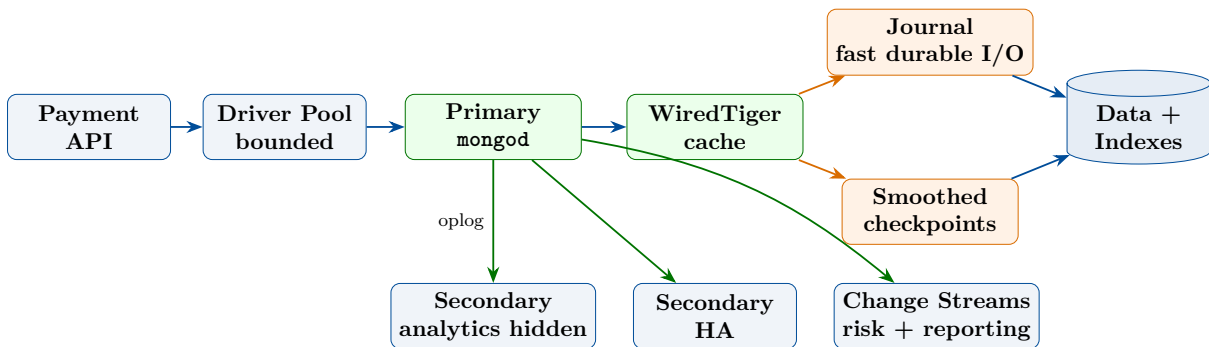


Figure 1.2: Financial ledger architecture balancing journal durability, WiredTiger cache, checkpoints, replica-set replication, and downstream change streams.

1.6.5 Operational Guardrails

Ledger architecture should include guardrails that stop accidental overload from becoming data risk. Use bounded driver pools, idempotent transaction identifiers, explicit write concern, alerting on replication lag, and tested restore procedures. Use load shedding when queues grow beyond the recovery point objective. Keep the primary workload narrow; send broad reporting to secondaries or downstream systems only when consistency requirements permit it.

Common Pitfall

Solving financial durability with storage tuning alone. Cache and checkpoint tuning cannot compensate for missing idempotency, unclear write concern, weak backup validation, or untested failover procedures.

1.7 Interview Q&A

1.7.1 Concepts and Trade-offs

1. **Q: What does WiredTiger cache, and why is it different from the OS page cache?**

A: WiredTiger caches database pages and update state inside `mongod`; the OS caches filesystem blocks outside the process. WiredTiger understands dirty pages and eviction, while the OS page cache understands files and blocks.

2. **Q: Why can a checkpoint create latency spikes?**

A: If dirty pages accumulate faster than storage can flush them, checkpoint work competes with foreground operations and journal activity. The symptom is often p99 latency, not average latency.

3. **Q: How do write concern and journaling differ?**

A: Journaling protects local crash recovery. Write concern defines acknowledgement requirements, including whether replication to other replica-set members is required.

4. **Q: Why are too many indexes dangerous for write-heavy workloads?**

A: Each index must be updated on writes, consumes cache, increases checkpoint work, and lengthens recovery and backup activity.

1.8 Further Reading

Start with MongoDB’s architecture, WiredTiger, write concern, replication, sharding, and `serverStatus` documentation [15, 26, 27, 21, 23, 22]. For storage-engine foundations, read the WiredTiger architecture guide [33], the original B-tree paper by Bayer and McCreight [4], and the ARIES write-ahead logging paper [11]. For database-category context, compare document and NoSQL systems using Stonebraker’s short critique and Han et al.’s survey [32, 7]. MongoDB Atlas Vector Search becomes relevant later when the storage and indexing foundations in this chapter are connected to AI search workloads [12].

Key Takeaways

- `mongod` coordinates clients, commands, query execution, replication roles, and storage-engine calls; WiredTiger owns page storage, cache, compression, journaling, and checkpoints.
- WiredTiger’s cache is not the same as the OS page cache. Production sizing must leave memory for both plus non-cache server overhead.
- B+Tree data and index files make locality, selectivity, and index count central to performance.
- Writes first change in-memory pages, then rely on journal and checkpoint mechanisms for durable recovery.
- Checkpoint I/O can create periodic tail-latency spikes when dirty pages, cache sizing, indexes, and disk bandwidth are misaligned.
- `db.serverStatus().wiredTiger` exposes the evidence needed to move from speculation to diagnosis.
- Financial ledger systems require explicit durability semantics, idempotency, bounded concurrency, right-sized cache, and downstream offload for reporting.

1.9 Exercises

1. **(Conceptual)** Draw the boundary between the MongoDB query layer and WiredTiger. Name three responsibilities on each side of the boundary.
2. **(Conceptual)** Explain why a larger WiredTiger cache can improve read locality while also making checkpoint spikes worse under some write-heavy workloads.
3. **(Conceptual)** Compare journaling with majority write concern. Which failure does each protect against, and where do they overlap?
4. **(Hands-on)** Run the lab with two different `cacheSizeGB` values. Record cache occupancy, dirty bytes, pages evicted, checkpoint time, and p95 insert latency.
5. **(Hands-on)** Modify the workload to add two extra secondary indexes. Measure how insert throughput and checkpoint metrics change.
6. **(Design)** Design a ledger document and index set for a tenant-aware payments platform. Defend every index using a query pattern and a write-path cost.
7. **(Design)** Create an alert policy for WiredTiger eviction pressure. Include at least one cache metric, one latency metric, and one disk metric.

8. **(Interview)** A team wants to set WiredTiger cache to 90% of host RAM because “MongoDB should use memory.” Write a concise architecture-review response explaining why this is risky.

1.10 Capstone Project: A WiredTiger Cache-Tuning Lab for a Financial Ledger

Capstone Project

Mission: build a repeatable Windows and PowerShell lab that proves how WiredTiger cache sizing changes write throughput, eviction pressure, checkpoint duration, journal activity, and p99 latency for a financial-ledger workload. You will run the same ledger workload under at least two cache configurations, collect comparable `db.serverStatus().wiredTiger` evidence, and recommend a cache setting that reduces checkpoint I/O spikes without starving the operating-system page cache.

Estimated time: 4–6 hours, depending on storage speed and whether the full 10 million document data set fits comfortably on your lab machine.

What you will practice:

- Connecting `mongod` process architecture to WiredTiger cache, eviction, journaling, checkpointing, and crash recovery.
- Configuring `storage.wiredTiger.engineConfig.cacheSizeGB` in a local Community Edition deployment.
- Loading a large immutable ledger collection and driving sustained write-heavy traffic.
- Capturing cache, eviction, journal, and checkpoint metrics before and after a cache change.
- Explaining a tuning recommendation with evidence instead of intuition.

Scenario

You are the data engineer on a payments platform that posts immutable financial-ledger entries for card authorizations, settlement events, reversals, and fee adjustments. During the Friday peak window, application logs show a regular pattern: insert latency is acceptable for most of each minute, then p99 latency jumps when checkpoint I/O competes with foreground writes. The business cannot weaken journaled durability, cannot drop the account-history index, and cannot move the primary ledger workload to an analytical replica during posting.

Your goal is to eliminate or materially reduce checkpoint I/O spikes by balancing the WiredTiger cache against the operating-system page cache. A cache that is too small creates eviction pressure and foreground stalls. A cache that is too large can let more dirty data accumulate, reduce memory available for filesystem caching, and produce larger checkpoint flushes. The best answer is the measured middle ground for this workload, this host, and this durability contract.

Requirements

- Use a local MongoDB Community `mongod` instance on Windows, started from PowerShell, with an explicit `storage.wiredTiger.engineConfig.cacheSizeGB` value in `mongod.conf`.
- Prepare at least two configurations, such as a small-cache run near 20% of available RAM and a larger-cache run near 50% of available RAM. Leave enough memory for Windows, the OS page cache, the `mongod` process outside WiredTiger, terminal tools, and monitoring.

- Generate 10 million ledger documents with immutable fields such as `accountId`, `transactionId`, `postedAt`, `amount`, `currency`, `direction`, and `sequence`. If the lab machine cannot safely hold 10 million documents, document the reduced count and keep the method identical.
- Create a ledger-style index, at minimum `{ accountId: 1, postedAt: -1 }`, and optionally a unique transaction index to test write-path cost.
- Run a write-heavy workload that inserts new ledger events while performing bounded account-history reads.
- Capture `db.serverStatus().wiredTiger` cache, eviction, log, transaction, and checkpoint metrics under each cache configuration [22].
- Analyze checkpoint stalls by comparing p95 and p99 insert latency, dirty bytes, pages evicted by application threads, checkpoint duration, bytes written, and journal sync behavior.
- Deliver a tuning recommendation that names the selected cache value, explains the trade-off with the OS page cache, and states what evidence would trigger a future retest.

Reference Architecture

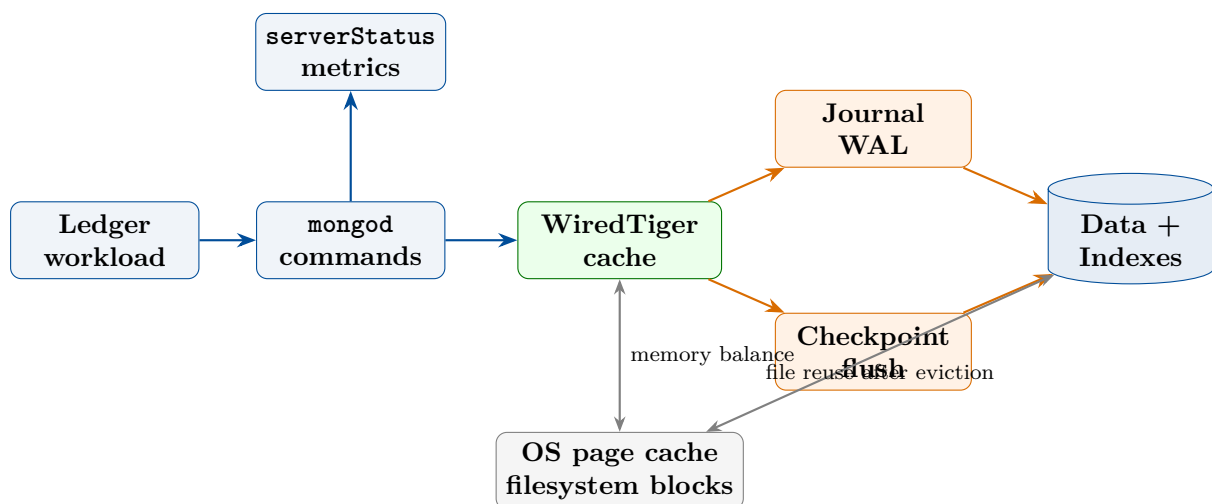


Figure 1.3: Capstone lab architecture for balancing WiredTiger cache, journal write-ahead logging, checkpoint flushing, durable files, and the operating-system page cache.

Implementation Steps

1. **Choose cache values.** Record physical RAM, available RAM before starting `mongod`, disk type, MongoDB version, and the two cache targets. Example: on a 32 GB dedicated lab machine, compare `cacheSizeGB: 6` with `cacheSizeGB: 16`. On an 8 GB laptop, use smaller values and reduce the document count if needed.
2. **Create a reusable lab folder.** Keep each run isolated so that logs, data files, and metrics do not overwrite each other. The following configuration is a template; copy it once per cache size and change `dbPath`, `path`, and `cacheSizeGB`.

```

1 storage:
2   dbPath: C:\MongoData\wt-ledger-cache20\db
3   journal:
4     enabled: true

```

```

5   wiredTiger:
6     engineConfig:
7       cacheSizeGB: 6
8     collectionConfig:
9       blockCompressor: snappy
10    indexConfig:
11      prefixCompression: true
12  systemLog:
13    destination: file
14    path: C:\MongoData\wt-ledger-cache20\log\mongod.log
15    logAppend: true
16  net:
17    bindIp: 127.0.0.1
18    port: 27017
19  processManagement:
20    timeZoneInfo: C:\Program Files\MongoDB\Server\8.0\share\zoneinfo

```

Listing 1.4: PowerShell lab mongod.conf with explicit WiredTiger cache sizing.

3. **Start one run at a time from PowerShell.** Use a foreground mongod terminal for the active run, or register a Windows service only if that is already your local standard. The command block uses PowerShell syntax but the listing style is the book's shell style.

```

1 $RunName = "wt-ledger-cache20"
2 $Root = "C:\MongoData\$RunName"
3 New-Item -ItemType Directory -Force "$Root\db", "$Root\log", "$Root\metrics"
4 mongod --config "$Root\mongod.conf"
5
6 # In a second PowerShell terminal after mongod is listening:
7 mongosh "mongodb://127.0.0.1:27017/ledger" --eval "db.runCommand({ ping: 1
  })"

```

Listing 1.5: PowerShell commands to prepare one cache-size run.

4. **Load 10 million ledger documents.** The script below creates deterministic batches, inserts immutable ledger events, and adds the core account-history index. Run it once for each isolated data directory. If storage is limited, lower `targetDocuments` but keep the same batch size and field shape.

```

1 use ledger;
2 db.entries.drop();
3
4 const targetDocuments = 10000000;
5 const batchSize = 10000;
6 const currencies = ["USD", "EUR", "MXN"];
7 const directions = ["debit", "credit"];
8
9 for (let base = 0; base < targetDocuments; base += batchSize) {
10   const batch = [];
11   for (let i = 0; i < batchSize; i++) {
12     const n = base + i;
13     batch.push({
14       accountId: "acct-" + (n % 250000).toString().padStart(6, "0"),
15       transactionId: "txn-" + n.toString().padStart(10, "0"),
16       postedAt: new Date(Date.now() - (n % 86400000)),
17       amount: NumberDecimal(((n % 200000) / 100 - 1000).toFixed(2)),
18       currency: currencies[n % currencies.length],
19       direction: directions[n % directions.length],
20       source: "capstone-load",
21       sequence: n
22     });
23   }
24 }

```

```

24 db.entries.insertMany(batch, { ordered: false, writeConcern: { w: 1, j:
    true } });
25 if (base % 100000 === 0) {
26   print("loaded " + base + " documents");
27 }
28 }
29
30 db.entries.createIndex({ accountId: 1, postedAt: -1 });
31 db.entries.createIndex({ transactionId: 1 }, { unique: true });
32
33 const wt = db.serverStatus().wiredTiger;
34 printjson({
35   phase: "after-load",
36   cache: wt.cache,
37   log: wt.log,
38   checkpoint: wt.checkpoint
39 });

```

Listing 1.6: mongosh data loader and baseline WiredTiger snapshot.

5. **Run a write-heavy workload and record metrics.** The Python script uses PyMongo so that latency can be measured client-side while WiredTiger counters are sampled from the server. Install PyMongo only if it is not already present in your lab environment. Save one metrics file per cache configuration.

```

1 python -m pip install pymongo
2 python .\MongoDB\book\scripts\wt_ledger_workload.py --uri mongoddb
   ://127.0.0.1:27017 --database ledger --seconds 600 --metrics C:\
   MongoData\wt-ledger-cache20\metrics\metrics.jsonl

```

Listing 1.7: PowerShell setup for the Python workload driver.

```

1 import argparse
2 import json
3 import random
4 import statistics
5 import time
6 from datetime import datetime, timezone
7
8 from pymongo import MongoClient, WriteConcern
9
10
11 def percentile(values, pct):
12     if not values:
13         return 0
14     ordered = sorted(values)
15     index = min(len(ordered) - 1, int(round((pct / 100) * (len(ordered) -
16     1))))
17     return ordered[index]
18
19 parser = argparse.ArgumentParser()
20 parser.add_argument("--uri", default="mongodb://127.0.0.1:27017")
21 parser.add_argument("--database", default="ledger")
22 parser.add_argument("--seconds", type=int, default=600)
23 parser.add_argument("--metrics", required=True)
24 args = parser.parse_args()
25
26 client = MongoClient(args.uri)
27 db = client[args.database]
28 entries = db.get_collection("entries", write_concern=WriteConcern(w=1, j=
    True))
29 accounts = [f"acct-{n:06d}" for n in range(250000)]

```



```

30 latencies_ms = []
31 end_time = time.time() + args.seconds
32 sequence = int(time.time())
33
34 with open(args.metrics, "a", encoding="utf-8") as metrics_file:
35     while time.time() < end_time:
36         account_id = random.choice(accounts)
37         document = {
38             "accountId": account_id,
39             "transactionId": f"capstone-{sequence}",
40             "postedAt": datetime.now(timezone.utc),
41             "amount": round(random.uniform(-2500, 2500), 2),
42             "currency": "USD",
43             "direction": "credit" if sequence % 2 == 0 else "debit",
44             "source": "capstone-workload",
45             "sequence": sequence,
46         }
47         started = time.perf_counter()
48         entries.insert_one(document)
49         if sequence % 100 == 0:
50             list(entries.find({"accountId": account_id}).sort("postedAt",
51 -1).limit(20))
52             latencies_ms.append((time.perf_counter() - started) * 1000)
53             sequence += 1
54
55         if len(latencies_ms) >= 1000:
56             status = db.command("serverStatus")["wiredTiger"]
57             row = {
58                 "capturedAt": datetime.now(timezone.utc).isoformat(),
59                 "insertP50Ms": percentile(latencies_ms, 50),
60                 "insertP95Ms": percentile(latencies_ms, 95),
61                 "insertP99Ms": percentile(latencies_ms, 99),
62                 "insertMeanMs": statistics.mean(latencies_ms),
63                 "cache": status.get("cache", {}),
64                 "log": status.get("log", {}),
65                 "transaction": status.get("transaction", {}),
66                 "checkpoint": status.get("checkpoint", {}),
67             }
68             metrics_file.write(json.dumps(row, default=str) + "\n")
69             metrics_file.flush()
70             latencies_ms.clear()

```

Listing 1.8: PyMongo workload driver that records latency and WiredTiger metrics.

6. **Repeat with the larger cache.** Stop mongod, start the second configuration, reload or restore the same baseline data shape, run the same duration, and write metrics to a separate file. Do not compare a cold first run with a warm second run unless you explicitly label that difference.
7. **Analyze checkpoint stalls.** Plot or tabulate each sample interval. Pay special attention to intervals where p99 latency rises at the same time as dirty bytes, checkpoint duration, checkpoint bytes written, or application-thread eviction counters increase. Use the categories in the MongoDB WiredTiger and `serverStatus` documentation as your field guide [26, 22].
8. **Write the recommendation.** The recommendation must include the chosen cache size, rejected alternatives, expected impact on checkpoint spikes, and at least two follow-up metrics to monitor in production.

Deliverables

- Two or more `mongod.conf` files, named by cache configuration, with data and log paths that prove each run was isolated.
- The data-loader script or `mongosh` transcript used to create the 10 million document ledger data set.
- The workload script and raw metrics files for every run.
- A short analysis document with the before-and-after metrics comparison table.

Table 1.1: Example before-and-after comparison for the cache-tuning recommendation.

Metric	Small about RAM	cache, 20%	Larger about RAM	cache, 50%	Interpretation
Configured <code>cacheSizeGB</code>					Confirm tested values.
Insert p99 latency					Tail-latency impact during checkpoints.
Bytes currently in cache					Working-set fit and cache headroom.
Tracked dirty bytes					Dirty-cache pressure before flush.
Pages evicted by application threads					Foreground eviction stall signal.
Checkpoint duration or time writing					Checkpoint spike shape.
Journal bytes and sync count					Durability path load.
Disk queue or write latency					Whether storage is the limiting layer.

Acceptance Criteria

- The lab runs from Windows PowerShell against a local MongoDB Community `mongod`.
- At least two explicit `cacheSizeGB` configurations are tested with comparable data volume, indexes, workload duration, and write concern.
- The ledger collection contains the required financial fields and either 10 million documents or a documented reduced count with a clear machine-capacity reason.
- Metrics include WiredTiger cache, eviction, log, transaction, and checkpoint sections from `db.serverStatus().wiredTiger`.
- The analysis identifies whether p99 latency spikes align with checkpoint activity, dirty-cache growth, or application-thread eviction.
- The final recommendation is justified by measurements and explicitly discusses the trade-off between WiredTiger cache and the OS page cache.
- The recommendation preserves the ledger durability contract; it does not disable journaling or weaken write concern merely to improve benchmark numbers.

- The submitted table is complete enough for another engineer to reproduce the conclusion.

Stretch Goals

- Vary WiredTiger eviction thresholds in a controlled development environment and observe whether foreground eviction decreases or merely moves the bottleneck.
- Compare collection compression with Snappy and Zstandard to see whether lower I/O offsets extra CPU during checkpoints.
- Test a different journal commit interval where supported by your MongoDB version and explain the durability and latency implications.
- Measure p95 and p99 read latency separately from insert latency, especially for recent account-history queries.
- Add two extra secondary indexes and quantify the write amplification they add to cache dirtiness and checkpoint work.
- Run a crash-recovery drill after a workload burst: stop the process abruptly in a lab, restart `mongod`, and record recovery time and log messages. Do not run this drill on shared or production data.
- Export Windows Performance Monitor counters for disk writes, disk queue length, CPU, and available memory, then correlate them with the WiredTiger samples.

Advanced Schema Design Patterns

Learning Objectives

- Explain how MongoDB schema design starts from query, update, and growth patterns rather than from third-normal-form decomposition.
- Apply the Polymorphic, Attribute, Bucket, and Outlier Patterns to collections that contain varied entities, dynamic fields, time-series events, and exceptional documents.
- Use the Computed, Subset, Extended Reference, and Schema Versioning Patterns to reduce hot-path reads, bound working-set memory, avoid repetitive lookups, and evolve documents safely.
- Design data-lifecycle models with Time-To-Live indexes, pre-allocation, and archival collections while preserving operational and compliance requirements.
- Refactor a monolithic e-commerce product catalog with the Attribute Pattern and Subset Pattern using `mongosh` and PyMongo examples.
- Architect a bucketed IoT metrics store for 100,000 events per second and prove the document-count and metadata-overhead reduction.

Cross-Cloud Equivalence

Document-model schema design appears in every cloud platform, but the modeling knobs differ. MongoDB patterns place related data in BSON documents and choose indexes around access paths; DynamoDB often uses single-table design with compound primary keys and overloaded item types; Cosmos DB for NoSQL emphasizes partition keys, item shape, and request-unit efficiency; Firestore and Datastore organize documents, collections, entity groups, and composite indexes. Relational services such as Amazon RDS, Azure SQL Database, and Cloud SQL usually normalize entities into tables and use joins, constraints, and materialized views for equivalent access paths.

Concept	MongoDB	DynamoDB	Cosmos DB	Firestore
Polymorphic entities	One collection with type discriminator and shared indexes	Single-table items with entity prefix in partition or sort key	Container with item type and partition key	Collection or collection group with type field
Dynamic attributes	Attribute Pattern array of key-value sub-documents	Sparse attributes or map fields projected into GSIs when needed	JSON properties with selective indexing policy	Map fields plus composite indexes for queried attributes
Time-series buckets	Bucket Pattern or time-series collection	Partition by device and time window; item per bucket	Partition by device or tenant plus windowed bucket item	Document per bucket or subcollection per device
Hot subset	Embedded recent subset in parent document	Duplicated summary item near access key	Denormalized hot properties in same logical partition	Parent document summary plus child collection for full history
References	Extended Reference with copied stable fields	Duplicated attributes in item families	Denormalized fields to avoid cross-partition fan-out	Duplicated reference fields to avoid extra reads
Lifecycle expiry	TTL index or Atlas Online Archive	TTL attribute	TTL policy	TTL policy or scheduled deletion
Relational equivalent	Embedded documents replace many joins	Access-pattern table replaces join model	JSON container trades joins for partition-local reads	Hierarchical documents replace selected joins

Snowflake and Microsoft Fabric commonly keep semi-structured data in **VARIANT** or **JSON**-capable columns and project it into relational views, tables, or lakehouse shortcuts. They are excellent analytical landing zones for flexible documents, but they do not remove the need to model MongoDB’s operational write path, indexes, shard keys, and working set correctly [17, 1, 8, 5, 31, 10].

2.1 Week 2 Roadmap: Designing Documents Around Workloads

Schema design in MongoDB is an engineering discipline, not a license to store arbitrary JSON forever. The central question is: *which document shape minimizes total cost for the dominant reads, writes, updates, retention rules, and growth modes?* The answer may embed data, reference data, duplicate stable fields, bucket events, compute summaries, or move cold records to archival storage. MongoDB’s data-modeling guidance starts from application query patterns and relationships because the document model can encode locality directly in the record [17, 13].

Definition

A **schema design pattern** is a repeatable document-shaping strategy that solves a common access-path, growth, lifecycle, or operational problem. Patterns are not mutually exclusive; a production collection may combine Attribute, Bucket, Computed, and Schema Versioning decisions in one design.

Key Terms

Embedding: storing related data inside the same document. **Referencing:** storing an identifier to another document. **Working set:** the subset of data and indexes repeatedly touched by active operations. **Cardinality:** the number of distinct values or related records. **Fan-out:** the number of documents or partitions a query must touch. **Write amplification:** extra writes caused by indexes, denormalized copies, summaries, or bucket

updates.

2.2 Monday: Core Design Patterns

2.2.1 Polymorphic Pattern

The Polymorphic Pattern stores related but not identical document types in one collection. It is useful when entities share enough access paths to justify common indexes but differ in optional fields. A media catalog, for example, may store books, videos, and podcasts together because users search by title, creator, tags, and publication date even though each media type has different metadata.

Definition

The **Polymorphic Pattern** uses a discriminator field, such as `kind` or `type`, so one collection can hold multiple document shapes while preserving queryable common fields.

Note

Polymorphism is strongest when shared predicates are real. If every query includes a different field set, one collection can become a weak compromise that carries too many sparse indexes.

Tip

Index common predicates first, then add partial indexes for type-specific paths. A partial index on `kind`: `"subscription"` can avoid making every document pay for a field used by only one subtype.

Common Pitfall

Hiding unrelated domains in one collection. A type field is not enough. Keep documents together only when they share lifecycle, security rules, scale profile, and important access paths.

2.2.2 Attribute Pattern

The Attribute Pattern converts many dynamic or sparsely populated fields into an array of name-value subdocuments. It is common in product catalogs, clinical observations, asset inventories, and machine specifications where each category has different attributes but users filter across a small set of attribute names.

Definition

The **Attribute Pattern** represents variable fields as entries such as `{ k: "screen_size", v: "15.6", unit: "inch" }` instead of creating a new top-level field for every possible attribute.

Tip

Create a compound multikey index on the attribute key and value only for attributes that are actually queried. For example, `{ "attributes.k": 1, "attributes.v": 1 }` supports filtering by attribute name and value.

Common Pitfall

Turning every field into an attribute. Stable, high-selectivity, commonly projected fields such as `tenantId`, `sku`, `status`, `category`, and `price` usually deserve top-level fields and ordinary indexes.

2.2.3 Bucket Pattern

The Bucket Pattern groups many small, related events into one document. It is especially valuable for time-series measurements because a single bucket can store metadata once and many readings in arrays or compact subdocuments. Instead of one document per reading, the collection stores one document per device per time window or count threshold.

Definition

The **Bucket Pattern** stores a bounded group of events in one document, typically keyed by entity and time window, so metadata is repeated once per bucket rather than once per event.

Note

MongoDB time-series collections automate parts of this idea, but data engineers still need to understand bucket boundaries, metadata fields, granularity, retention, and query patterns [24].

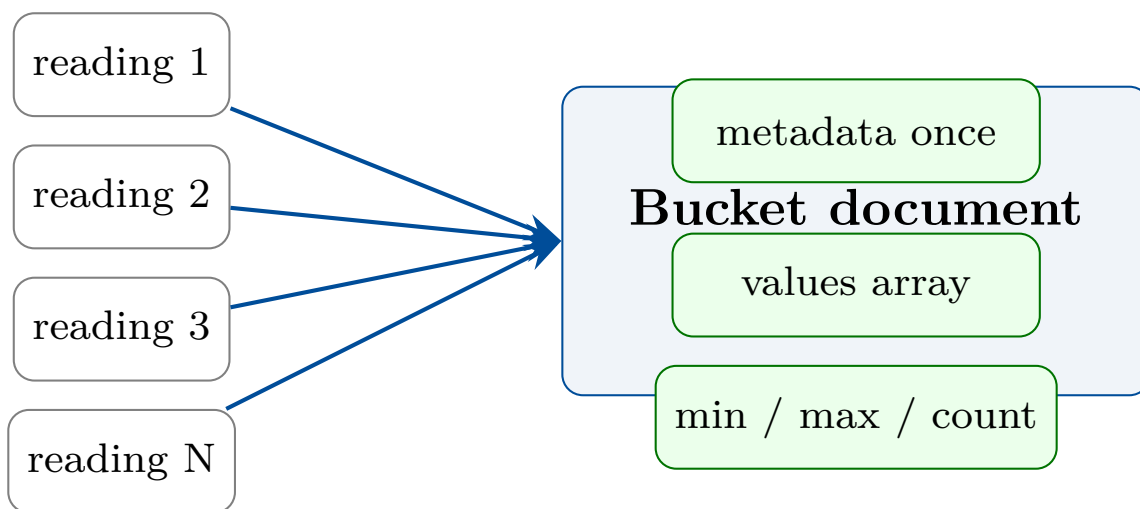


Figure 2.1: Bucket Pattern grouping many time-series readings into one bounded document with shared metadata and summary fields.

Common Pitfall

Unbounded buckets. Buckets must close by time, count, or size. A document that grows forever risks large updates, jumbo chunks in sharded clusters, and the 16 MB document limit.

2.2.4 Outlier Pattern

The Outlier Pattern handles rare documents whose size or relationship count would distort the common schema. Most users may have a small number of permissions, addresses, or notifica-

tions, while a few enterprise accounts have thousands. The normal case stays embedded for locality; the exceptional case spills to a side collection.

Definition

The **Outlier Pattern** keeps the common case simple and adds an overflow path for documents that exceed a defined threshold.

Tip

Store an explicit marker such as `hasOutlierItems: true` and a count. Queries can fetch the normal embedded array first and follow the overflow reference only when needed.

Common Pitfall

Designing the whole model for the rarest customer. If 0.1% of accounts have extreme cardinality, do not force every account to pay the read cost of a fully normalized design.

2.3 Tuesday: Operational Patterns

2.3.1 Computed Pattern

The Computed Pattern stores derived values that are expensive or repetitive to calculate on every request. Examples include order totals, product review averages, inventory availability, per-device hourly summaries, and dashboard counters.

Definition

The **Computed Pattern** materializes derived values in a document so the hot read path avoids repeated aggregation work.

Note

A computed value is a consistency decision. Some systems update it transactionally with the source write; others accept eventual consistency through change streams, scheduled jobs, or aggregation pipelines with `$merge`.

Tip

Record the source and freshness of computed fields with names such as `stats.updatedAt` or `rollupVersion`. This makes stale summaries observable.

Common Pitfall

Materializing everything. Computed fields reduce read cost but add write work and correctness obligations. Compute only values with clear read-frequency or latency justification.

2.3.2 Subset Pattern

The Subset Pattern embeds a small hot subset of related data in the parent document while keeping the complete set elsewhere. A product page may embed the first three reviews, a user profile may embed recent notifications, and an account may embed recent transactions. Reads

for the common screen stay fast and memory-resident; deep history remains available through a separate collection.

Definition

The **Subset Pattern** stores the most frequently read related records with the parent document and stores the complete related collection separately.

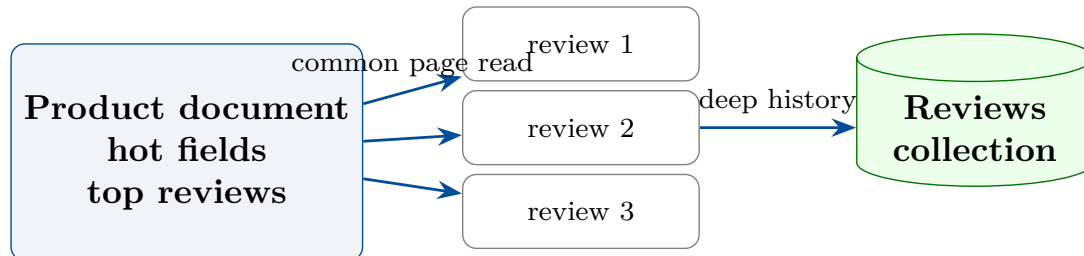


Figure 2.2: Subset Pattern keeping a small hot subset near the parent document while preserving full history in a separate collection.

Common Pitfall

Forgetting synchronization. The subset must be updated when the full collection changes. Decide whether updates are transactional, event-driven, or repaired by a periodic reconciliation job.

2.3.3 Extended Reference Pattern

The Extended Reference Pattern copies stable fields from a referenced document into the referencing document. An order may store `customerId`, `customerName`, and loyalty tier at purchase time. A support ticket may store `accountId`, account name, and service plan to avoid a join on every ticket list page.

Definition

The **Extended Reference Pattern** stores the foreign key plus selected stable descriptive fields from the referenced entity to avoid extra reads in common paths.

Tip

Copy fields that are stable or historically meaningful. A shipping order should keep the customer name used at order time even if the current profile changes later.

Common Pitfall

Copying volatile fields. Duplicating frequently changing fields creates fan-out updates. If the field changes often and every copy must be current, reference it instead or maintain it through a controlled update workflow.

2.3.4 Schema Versioning Pattern

Schema Versioning allows documents to evolve gradually without requiring a risky all-at-once migration. Each document carries a version field, and application code knows how to read old and new shapes during the transition.

Definition

The **Schema Versioning Pattern** records document shape with a field such as `schemaVersion`, allowing readers, writers, and migrations to coexist while data evolves.

Note

Schema versioning is not an excuse for permanent chaos. Keep the number of supported versions small, document migration rules, and remove old reader branches after the migration is complete.

Tip

Prefer forward-only migrations with idempotent update scripts. Store enough evidence to resume safely after interruption, especially for collections with millions of documents.

2.4 Wednesday: Data Lifecycle Patterns

2.4.1 Time-To-Live Pattern

TTL indexes automatically delete documents after a date field reaches a configured age. They are useful for sessions, expiring invitations, short-lived telemetry, cache collections, and privacy retention windows.

Definition

A **TTL index** is a single-field index with `expireAfterSeconds`; MongoDB's background monitor removes expired documents asynchronously [25].

Tip

Use TTL for data that can be deleted eventually, not exactly at a contractual second. The TTL monitor runs periodically, and deletion time can lag behind expiration time.

Common Pitfall

Using TTL where archival is required. TTL deletes data. If compliance or analytics requires retention, move documents to an archive first, then expire the operational copy.

2.4.2 Pre-allocation Pattern

Pre-allocation creates a known document or bucket structure before all data arrives. It can simplify high-rate updates when the target bucket is predictable, such as one IoT device per minute or one calendar slot per facility. Modern WiredTiger does not require manual padding in the way older storage engines did, so the pattern should be used for logical predictability, not folklore about physical allocation.

Definition

Pre-allocation creates bounded placeholder documents or arrays in advance so writers update known records instead of repeatedly discovering or creating the target shape.

Note

Pre-allocation is most useful when it reduces contention and branching in application code. It can be harmful if many empty buckets are created and never filled.

2.4.3 Archival Pattern

The Archival Pattern moves cold data out of the operational collection while preserving queryability through a separate collection, Atlas Online Archive, object storage, or an analytical platform. The goal is to reduce working-set pressure without losing retention history.

Definition

The **Archival Pattern** separates hot operational data from cold historical data using a controlled copy, validation, and deletion process.

Tip

Make archives provable. Record counts, hash totals where practical, date ranges, and restore tests. A retention process that cannot prove what moved is an operational risk.

Common Pitfall

Archiving without indexes on restore paths. Cold data eventually becomes important during audits, incidents, or customer disputes. Design the archive query keys before deleting the hot copy.

2.5 Thursday Hands-On Project: Refactor a Product Catalog**2.5.1 Scenario**

A monolithic e-commerce product document has grown organically. Every category adds new top-level fields, and the product page embeds every review, all media objects, warehouse details, and vendor-specific specifications. The result is wide documents, many sparse indexes, high memory pressure, and slow category filters. We will refactor the shape using the Attribute Pattern for dynamic specifications and the Subset Pattern for product-page reviews.

```

1 use catalog;
2 db.products.insertOne({
3   _id: ObjectId("64f00000000000000000101"),
4   sku: "LAP-2026-15-OLED",
5   category: "laptops",
6   title: "15 inch OLED developer laptop",
7   brand: "Northwind",
8   price: NumberDecimal("1499.00"),
9   screen_size_in: 15.6,
10  screen_type: "OLED",
11  cpu_model: "Ryzen AI 9",
12  memory_gb: 32,
13  storage_gb: 1000,
14  keyboard_layout: "US",
15  battery_wh: 84,
16  shoe_size: null,
17  fabric: null,
18  reviews: [
19    { reviewId: "r1", rating: 5, title: "fast", createdAt: ISODate("2026-07-01")
    } ],

```

```

20     { reviewId: "r2", rating: 4, title: "bright screen", createdAt: ISODate("
21       2026-07-03") }
22   ],
23   warehouseAvailability: [
24     { warehouseId: "den-01", qty: 15 },
25     { warehouseId: "phx-02", qty: 8 }
26   ]
27 });

```

Listing 2.1: Before: monolithic product document with sparse dynamic fields and unbounded reviews.

The category-specific fields are sparse across the catalog. Indexing each one independently would inflate index storage and write cost. The reviews array also grows without bound, even though the product page needs only a few recent or most helpful reviews.

```

1 db.products.replaceOne(
2   { sku: "LAP-2026-15-OLED" },
3   {
4     sku: "LAP-2026-15-OLED",
5     schemaVersion: 2,
6     category: "laptops",
7     title: "15 inch OLED developer laptop",
8     brand: "Northwind",
9     price: NumberDecimal("1499.00"),
10    attributes: [
11      { k: "screen_size", v: 15.6, unit: "inch", n: 15.6 },
12      { k: "screen_type", v: "OLED" },
13      { k: "cpu_model", v: "Ryzen AI 9" },
14      { k: "memory", v: 32, unit: "GB", n: 32 },
15      { k: "storage", v: 1000, unit: "GB", n: 1000 },
16      { k: "keyboard_layout", v: "US" }
17    ],
18    reviewSummary: { count: 248, averageRating: 4.7, updatedAt: new Date() },
19    topReviews: [
20      { reviewId: "r1", rating: 5, title: "fast", createdAt: ISODate("
21        2026-07-01") },
22      { reviewId: "r2", rating: 4, title: "bright screen", createdAt: ISODate("
23        2026-07-03") }
24    ],
25    warehouseAvailability: [
26      { warehouseId: "den-01", qty: 15 },
27      { warehouseId: "phx-02", qty: 8 }
28    ]
29  },
30  { upsert: true }
31 );
32
33 db.products.createIndex({ category: 1, "attributes.k": 1, "attributes.v": 1 });
34 db.reviews.createIndex({ sku: 1, helpfulVotes: -1, createdAt: -1 });

```

Listing 2.2: After: Attribute Pattern for dynamic specifications and Subset Pattern for hot reviews.

```

1 from decimal import Decimal
2 from pymongo import MongoClient, UpdateOne
3
4 client = MongoClient("mongodb://127.0.0.1:27017")
5 db = client.catalog
6 products = db.products
7 reviews = db.reviews
8
9 ATTRIBUTE_FIELDS = {

```

```

10     "screen_size_in": ("screen_size", "inch"),
11     "screen_type": ("screen_type", None),
12     "cpu_model": ("cpu_model", None),
13     "memory_gb": ("memory", "GB"),
14     "storage_gb": ("storage", "GB"),
15     "keyboard_layout": ("keyboard_layout", None),
16 }
17
18 for old in products.find({"schemaVersion": {"$ne": 2}}):
19     attrs = []
20     for old_name, (new_name, unit) in ATTRIBUTE_FIELDS.items():
21         value = old.get(old_name)
22         if value is None:
23             continue
24         entry = {"k": new_name, "v": value}
25         if unit:
26             entry["unit"] = unit
27         if isinstance(value, (int, float, Decimal)):
28             entry["n"] = float(value)
29         attrs.append(entry)
30
31     full_reviews = old.get("reviews", [])
32     if full_reviews:
33         reviews.bulk_write([
34             UpdateOne(
35                 {"sku": old["sku"], "reviewId": review["reviewId"]},
36                 {"$set": {**review, "sku": old["sku"]}},
37                 upsert=True,
38             )
39             for review in full_reviews
40         ])
41
42     products.update_one(
43         {"_id": old["_id"]},
44         {"$set": {
45             "schemaVersion": 2,
46             "attributes": attrs,
47             "topReviews": sorted(full_reviews, key=lambda r: r.get("rating", 0)
48             , reverse=True)[:3],
49             "reviewSummary.count": len(full_reviews),
50             }, "$unset": {field: "" for field in ATTRIBUTE_FIELDS} | {"reviews": ""
51         }},
52     )

```

Listing 2.3: PyMongo migration sketch that writes the new product shape and full review collection.

Note

The migration is intentionally a sketch. A production migration would batch writes, record progress, handle retries idempotently, validate counts, and run behind feature flags or dual readers.

2.6 Friday Use Case and Architecture: IoT Metrics at 100,000 Events per Second

2.6.1 Scenario and Requirements

An industrial IoT platform receives metrics from 500,000 devices deployed across factories, vehicles, and edge gateways. Peak ingest reaches 100,000 events per second. Each event contains

tenant, device, sensor type, timestamp, value, unit, firmware version, and location metadata. If the platform stores one document per reading, it writes 8.64 billion documents per day and repeats metadata billions of times.

The Bucket Pattern changes the economics. Suppose each bucket stores 100 readings for one device, sensor, and minute. Document count drops from 8.64 billion event documents per day to about 86.4 million bucket documents per day, a 99% count reduction. If the metadata portion is roughly the same size as one reading payload, storing it once per 100 readings reduces metadata overhead by about 99%; with smaller bucket sizes or richer per-reading values, a conservative architecture claim of about 95% metadata reduction is realistic and easier to defend.

2.6.2 Bucketed Document Shape

A bucket document should contain immutable identity fields, a bounded time window, compact readings, summary values, and close-state metadata. Common query patterns include device timeline, tenant outage investigation, sensor percentile analysis, and downsampling into analytical stores.

```

1 {
2   tenantId: "tenant-acme",
3   deviceId: "press-17",
4   sensor: "temperature",
5   bucketStart: ISODate("2026-07-31T07:37:00Z"),
6   bucketEnd: ISODate("2026-07-31T07:38:00Z"),
7   count: 100,
8   min: 72.4,
9   max: 79.8,
10  sum: 7592.1,
11  metadata: {
12    firmware: "8.2.1",
13    siteId: "den-factory-03",
14    lineId: "line-4",
15    unit: "C"
16  },
17  readings: [
18    { offsetMs: 0, v: 73.1, quality: "ok" },
19    { offsetMs: 600, v: 73.3, quality: "ok" },
20    { offsetMs: 1200, v: 73.8, quality: "ok" }
21  ],
22  closed: true
23 }
```

Listing 2.4: Bucketed IoT metrics document for one device, sensor, and one-minute window.

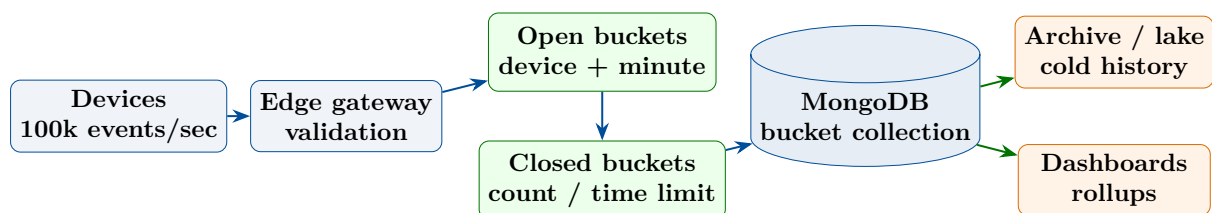


Figure 2.3: Bucketed IoT metrics architecture reducing document count and repeated metadata for 100,000 events per second.

Tip

Use bucket keys that align with both ingest and query: tenant, device, sensor, and time window. If sharded, choose a shard key that spreads write load without making device timeline queries scatter across every shard.

Common Pitfall

Hot-bucket contention. If too many writers update the same bucket concurrently, the bucket document becomes a hot record. Partition buckets by device, sensor, and short windows, or buffer at the edge before writing.

2.7 Interview Q&A

2.7.1 Concepts and Trade-offs

1. **Q: How do you decide whether to embed or reference?**
A: Embed when data is read together, bounded, and shares lifecycle. Reference when relationships are large, independently updated, independently secured, or queried on their own.
2. **Q: Why does the Attribute Pattern help product catalogs?**
A: It turns category-specific sparse fields into a consistent key-value array, enabling shared indexes for dynamic filters without creating many top-level sparse indexes.
3. **Q: What is the main risk of the Bucket Pattern?**
A: Unbounded or highly contended bucket documents. Buckets must close by count, time, or size, and write distribution must avoid hot records.
4. **Q: When is the Computed Pattern appropriate?**
A: When a derived value is read frequently enough that materializing it is cheaper than recomputing it, and the system has an acceptable consistency strategy.
5. **Q: How does Schema Versioning reduce migration risk?**
A: It lets readers and writers handle old and new document shapes during a rolling migration, so the database does not require a single blocking rewrite.
6. **Q: Why is TTL not the same as archival?**
A: TTL deletes expired documents asynchronously. Archival copies and validates cold data before removing it from the hot operational store.

2.8 Further Reading

Start with MongoDB's data modeling, schema pattern, time-series, TTL, and indexing documentation [17, 13, 24, 25, 18]. For adjacent platform modeling, compare DynamoDB single-table design, Cosmos DB partitioning, and Firestore data modeling [1, 8, 5]. For analytical handling of flexible documents, review Snowflake semi-structured data and Microsoft Fabric JSON guidance [31, 10].

Key Takeaways

- MongoDB schema design is workload design: query paths, update frequency, lifecycle, cardinality, and working set drive the document shape.

- The Polymorphic Pattern is useful for related entity types with shared access paths, while the Attribute Pattern handles dynamic searchable fields.
- The Bucket Pattern reduces document count and repeated metadata for time-series workloads, but buckets must be bounded and protected from hot-record contention.
- The Outlier Pattern keeps the common case embedded while moving rare high-cardinality cases to overflow collections.
- Computed, Subset, Extended Reference, and Schema Versioning Patterns trade extra write complexity for faster reads, lower memory pressure, and safer evolution.
- TTL, pre-allocation, and archival patterns manage data lifecycle, but deletion, retention, and restore requirements must be explicit.
- For IoT ingest at 100,000 events per second, bucketed schemas can cut document count and repeated metadata by about 95% or more when bucket sizes are well chosen.

2.9 Exercises

1. **(Conceptual)** Choose embed or reference for customer addresses, order line items, payment attempts, and product reviews. Explain the lifecycle and cardinality reason for each choice.
2. **(Conceptual)** Design a Polymorphic Pattern collection for digital media. Name the discriminator field, common indexes, and one partial index.
3. **(Hands-on)** Convert five category-specific product fields into Attribute Pattern entries and write the compound multikey index needed for filtering.
4. **(Hands-on)** Create a TTL index for expiring session documents after seven days. Explain why deletion may not occur at the exact expiration second.
5. **(Design)** Design an Outlier Pattern for enterprise accounts with more than 1,000 team members while keeping small accounts embedded.
6. **(Design)** Build a Schema Versioning migration plan from `schemaVersion: 1` to `schemaVersion: 2`; include reader behavior, writer behavior, and rollback evidence.
7. **(Performance)** Estimate daily document counts for one-event-per-document versus 100-event buckets at 100,000 events per second. State the operational impact on indexes and backups.
8. **(Interview)** A team wants every product specification as a top-level indexed field. Write a concise architecture-review response explaining the Attribute Pattern alternative.

2.10 Capstone Project: Design and Validate a Bucket-Pattern IoT Metrics Store

Capstone Project

Mission: design, implement, and validate a MongoDB IoT metrics store that uses the Bucket Pattern to reduce document count, repeated metadata, and working-set memory. You will model raw events, create bucket documents, load synthetic measurements, compare document counts against a one-event-per-document baseline, and prove the reduction with queries and metrics.

Estimated time: 4–6 hours, depending on local MongoDB speed and the event volume selected for the lab.

What you will practice:

- Translating event-rate and metadata assumptions into bucket size, bucket window, and index choices.
- Writing Windows PowerShell-friendly `mongosh` and PyMongo loaders.
- Creating bounded bucket documents with count, min, max, sum, and metadata fields.
- Measuring document-count reduction and estimating metadata savings.
- Defending a production architecture for high-rate telemetry ingestion.

Scenario

You own the telemetry store for a manufacturing analytics platform. Each device emits temperature, vibration, pressure, and power metrics. The platform must ingest a peak of 100,000 events per second, retain hot operational data for seven days in MongoDB, archive cold buckets to analytical storage, and support recent device timeline queries. The business wants proof that bucketed storage materially reduces document count and memory pressure before approving the architecture.

Requirements

- Model the one-event-per-document baseline and the bucketed design for the same tenant, device, sensor, and time range.
- Use buckets bounded by device, sensor, and one-minute windows, with an additional maximum reading count such as 100 or 500.
- Store metadata once per bucket: tenant, site, device, sensor, unit, firmware, and bucket time range.
- Store compact readings with timestamp offset and value, plus optional quality code.
- Maintain computed fields: `count`, `min`, `max`, `sum`, and `closed`.
- Create indexes for device timeline reads and archival scans, such as { `tenantId`: 1, `deviceId`: 1, `sensor`: 1, `bucketStart`: -1 }.
- Calculate expected document-count reduction and metadata-overhead reduction for the chosen bucket size.
- Validate the result by loading a repeatable sample and comparing collection counts and average document sizes.

Reference Architecture

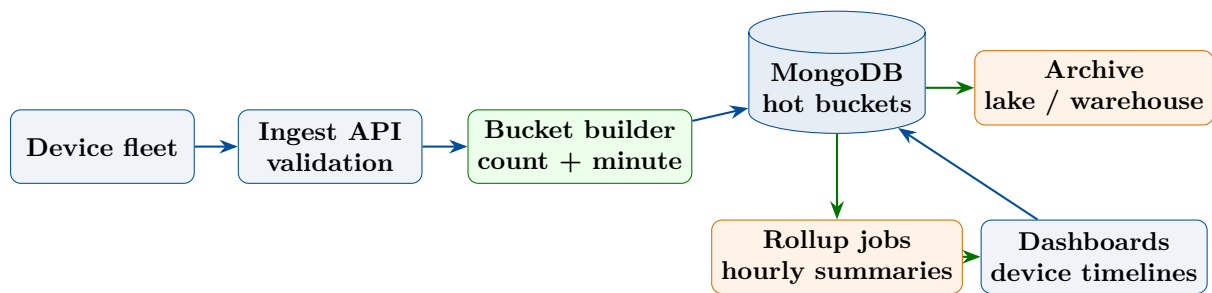


Figure 2.4: Capstone architecture for validating bucketed IoT metrics, hot MongoDB retention, rollups, and archival flow.

Implementation Steps

1. **Prepare the collections and indexes.** Use a local MongoDB database named `iot_capstone`. Keep raw and bucketed collections side by side so counts are comparable.

```

1 use iot_capstone;
2 db.raw_events.drop();
3 db.metric_buckets.drop();
4
5 db.raw_events.createIndex({ tenantId: 1, deviceId: 1, sensor: 1, ts: -1 });
6 db.metric_buckets.createIndex({ tenantId: 1, deviceId: 1, sensor: 1,
7   bucketStart: -1 });
8 db.metric_buckets.createIndex({ tenantId: 1, bucketStart: 1, closed: 1 });

```

Listing 2.5: mongosh setup for raw and bucketed IoT collections.

2. **Load a repeatable sample.** The script below keeps the sample small enough for a laptop but preserves the same math used for 100,000 events per second.

```

1 from datetime import datetime, timedelta, timezone
2 from pymongo import MongoClient, UpdateOne
3
4 client = MongoClient("mongodb://127.0.0.1:27017")
5 db = client.iot_capstone
6 raw = db.raw_events
7 buckets = db.metric_buckets
8 raw.delete_many({})
9 buckets.delete_many({})
10
11 start = datetime(2026, 7, 31, 7, 37, tzinfo=timezone.utc)
12 tenants = ["tenant-acme"]
13 devices = [f"press-{i:03d}" for i in range(100)]
14 sensors = ["temperature", "vibration", "pressure", "power"]
15 events_per_series = 250
16 bucket_size = 100
17 raw_batch = []
18 bucket_ops = []
19
20 for tenant in tenants:
21     for device in devices:
22         for sensor in sensors:
23             for i in range(events_per_series):
24                 ts = start + timedelta(milliseconds=i * 600)
25                 value = 70 + (i % 30) * 0.1
26                 raw_batch.append({

```

```

27         "tenantId": tenant,
28         "deviceId": device,
29         "sensor": sensor,
30         "ts": ts,
31         "value": value,
32         "unit": "C" if sensor == "temperature" else "raw",
33         "firmware": "8.2.1",
34         "siteId": "den-factory-03",
35     })
36     bucket_number = i // bucket_size
37     bucket_start = start + timedelta(minutes=bucket_number)
38     bucket_id = f"{tenant}|{device}|{sensor}|{bucket_start}.
isoformat()}"
39     bucket_ops.append(UpdateOne(
40         {"_id": bucket_id},
41         {
42             "$setOnInsert": {
43                 "tenantId": tenant,
44                 "deviceId": device,
45                 "sensor": sensor,
46                 "bucketStart": bucket_start,
47                 "bucketEnd": bucket_start + timedelta(minutes
=1),
48                 "metadata": {"firmware": "8.2.1", "siteId": "
den-factory-03"},
49                 "closed": False,
50             },
51             "$push": {"readings": {"offsetMs": int((ts -
bucket_start).total_seconds() * 1000), "v": value}},
52             "$inc": {"count": 1, "sum": value},
53             "$min": {"min": value},
54             "$max": {"max": value},
55         },
56         upsert=True,
57     ))
58
59 raw.insert_many(raw_batch, ordered=False)
60 buckets.bulk_write(bucket_ops, ordered=False)
61 buckets.update_many({"count": {"$gte": bucket_size}}, {"$set": {"closed":
True}})
62
63 raw_count = raw.count_documents({})
64 bucket_count = buckets.count_documents({})
65 print({
66     "rawCount": raw_count,
67     "bucketCount": bucket_count,
68     "documentReductionPercent": round((1 - bucket_count / raw_count) * 100,
2),
69 })

```

Listing 2.6: PyMongo loader that writes raw events and equivalent bucket documents.

3. **Measure size and query behavior.** Run collection stats and a representative device timeline query. Compare document counts, average object size, total index size, and the number of documents examined.

```

1 use iot_capstone;
2 const rawStats = db.raw_events.stats();
3 const bucketStats = db.metric_buckets.stats();
4 printjson({
5     rawCount: db.raw_events.countDocuments(),
6     bucketCount: db.metric_buckets.countDocuments(),
7     rawAvgObjSize: rawStats.avgObjSize,

```

```

8   bucketAvgObjSize: bucketStats.avgObjSize,
9   rawIndexSize: rawStats.totalIndexSize,
10  bucketIndexSize: bucketStats.totalIndexSize
11 });
12
13 printjson(db.metric_buckets.find({
14   tenantId: "tenant-acme",
15   deviceId: "press-017",
16   sensor: "temperature",
17   bucketStart: { $gte: ISODate("2026-07-31T07:37:00Z") }
18 }).sort({ bucketStart: 1 }).explain("executionStats"));

```

Listing 2.7: mongosh validation queries for count, size, and timeline access.

4. **Scale the math to production.** For 100,000 events per second, compute daily raw documents as $100000 * 86400$. Divide by the chosen bucket size to estimate bucket documents per day. Estimate metadata reduction as $1 - (1 / \text{bucketSize})$ when metadata is stored once per bucket instead of once per event, then temper the claim based on actual measured object sizes.

Deliverables

- A short schema design document showing raw event shape, bucket document shape, bucket key, close rule, and indexes.
- The `mongosh` setup script and PyMongo loader used in the lab.
- A results table with raw count, bucket count, reduction percentage, average object sizes, total collection sizes, and total index sizes.
- An explanation of how the sample maps to 100,000 events per second in production.
- A risk section covering hot buckets, late events, duplicate events, shard key choice, TTL, and archival.

Acceptance Criteria

- The bucket design stores metadata once per bucket and readings in bounded arrays.
- Buckets close by count and time window, preventing unbounded document growth.
- The validation load creates both raw and bucketed versions of the same logical events.
- The measured document-count reduction is reported and the production-scale calculation is shown.
- The design includes indexes for recent device timeline reads and archival scans.
- The architecture explains how hot seven-day data remains in MongoDB while cold data moves to archive.
- The recommendation discusses shard-key implications for high ingest and targeted reads.
- The final write-up proves or rejects the about 95% reduction claim with measured data and explicit assumptions.

Stretch Goals

- Add deduplication with an event identifier and prove duplicate writes do not inflate bucket counts.

- Support late-arriving events by reopening recent buckets or writing correction buckets.
- Compare bucket sizes of 50, 100, and 500 readings and measure update contention, query latency, and index size.
- Add a TTL index for hot buckets after archival validation and explain the deletion lag.
- Implement hourly rollup documents with the Computed Pattern and compare dashboard query cost.
- Test a sharded design in a development cluster and compare shard distribution for candidate shard keys.
- Export bucketed data to Parquet or JSON for Snowflake or Fabric analysis and document the field mapping.

Modeling Relationships and Modern Anti-Patterns

Learning Objectives

- Choose embedding or referencing for 1:1, 1:N few, 1:N many, and N:M relationships based on read locality, update frequency, lifecycle, and growth bounds.
- Recognize document anti-patterns such as unbounded arrays, bloated documents near the 16 MB BSON limit, and massive collections without lifecycle or partitioning strategy.
- Use MongoDB JSON Schema validation to enforce required fields, data types, enum rules, nested structures, and operational guardrails.
- Build an enterprise HR collection with `$jsonSchema` validation in `createCollection` and verify accepted and rejected documents with PyMongo.
- Re-architect a legacy user-activity logging system that fails with `BSONObj size exceeds 16MB` errors.
- Combine parent-referencing, capped collections, indexes, retention windows, and validation rules into a defensible production remediation plan.

Cross-Cloud Equivalence

Relationship modeling and referential integrity appear in every data platform, but each system exposes different knobs. MongoDB optimizes for locality through embedding and targeted references, with `$lookup` available when a read path needs server-side joining. Relational services such as Amazon RDS, Azure SQL Database, and Cloud SQL enforce foreign keys and execute joins natively. DynamoDB commonly models relationships with single-table adjacency items and application-enforced integrity. Cosmos DB for NoSQL favors embedding inside a logical partition and carefully chosen partition keys to avoid cross-partition fan-out.

Concept	MongoDB	Relational / Azure SQL / Cloud SQL	RDS / DynamoDB	Cosmos DB / NoSQL	for
1:1 relationship	Embed when read and lifecycle are shared; reference when independently secured or updated	Same table for optional columns or separate table with unique foreign key	Same item or adjacent item with same partition key	Embed in item when partition-local; split item if growth or access differs	split or access differs
1:N few	Embed bounded child array such as addresses or current roles	Child table with foreign key and join	Item collection under one partition key with sort-key prefixes	Embedded if bounded and partition-local	array and
1:N many	Parent-reference children; query by parent key; avoid unbounded parent arrays	Child table indexed by foreign key; pagination through joins or covering indexes	Many child items under partition key, often time-ordered sort key	Child items in same partition when possible; separate container for extreme scale	when separate
N:M relationship	Junction collection with both foreign keys, optional extended references, and indexes	Junction table with two foreign keys and uniqueness constraints	Edge items for both traversal directions in single-table design	Edge documents, often duplicated by traversal direction and partition key	documents, duplicated by traversal direction and partition key
Referential integrity	Usually application-enforced; transactions can coordinate bounded multi-document changes; <code>\$lookup</code> does not enforce integrity	Database-enforced foreign keys, cascading rules, constraints, and join optimizer	Application-enforced conditional writes and transactions for small item groups	Application-enforced; transactional batch only within logical partition	batch only within logical partition
Document / row size limits	16 MB BSON document limit; design arrays and embedded histories with hard bounds	Row and page limits vary by engine; large values often stored off-row or in separate tables	400 KB item limit encourages narrow items and adjacency records	2 MB item limit in API for NoSQL requires partition-aware item sizing	in API for NoSQL requires partition-aware item sizing
The architectural equivalence is not <i>documents versus joins</i> ; it is <i>where relationship cost is paid</i> . MongoDB pays design attention up front by placing bounded data together and referencing high-cardinality data. Relational systems pay at query time through joins backed by constraints. DynamoDB and Cosmos DB pay through key design, duplication, and partition-local access patterns [17, 20, 14, 1, 8].					

3.1 Week 3 Roadmap: Relationships, Limits, and Validation

MongoDB lets data engineers encode relationships directly inside documents, but that power is safe only when growth is bounded and access paths are explicit. A relationship that begins as a convenient embedded array can become a production incident when the array grows forever, every update rewrites a large parent document, and the record approaches the 16 MB BSON document limit [20]. This week focuses on designing relationship boundaries before they become operational failures.

Definition

Relationship modeling is the practice of deciding whether related facts belong in the same document, in separate documents connected by references, or in a hybrid design with selected duplicated fields.

Key Terms

1:1: one parent relates to one child. **1:N few**: one parent relates to a small bounded child set. **1:N many**: one parent relates to a large or unbounded child set. **N:M**: many records on both sides relate through an edge or junction. **Parent referencing**: each child stores the parent identifier. **Child referencing**: the parent stores child identifiers. **Cardinality**: relationship count. **Validation**: database-side rules that reject invalid document shapes.

3.2 Monday: Embedding versus Referencing

3.2.1 Decision Criteria

Embedding and referencing are not moral choices; they are workload choices. Embed when the child data is needed with the parent, changes at the same pace, has the same security boundary, and has a clear maximum size. Reference when the child data is large, independently updated, independently retained, or queried without the parent.

Definition

Embedding stores related data inside the same BSON document. **Referencing** stores an identifier, such as `employeeId` or `departmentId`, that points to a document in the same or another collection.

Note

The important question is not whether MongoDB can perform a `$lookup`. It can. The important question is whether the dominant access path benefits from locality or from independent document growth and indexing.

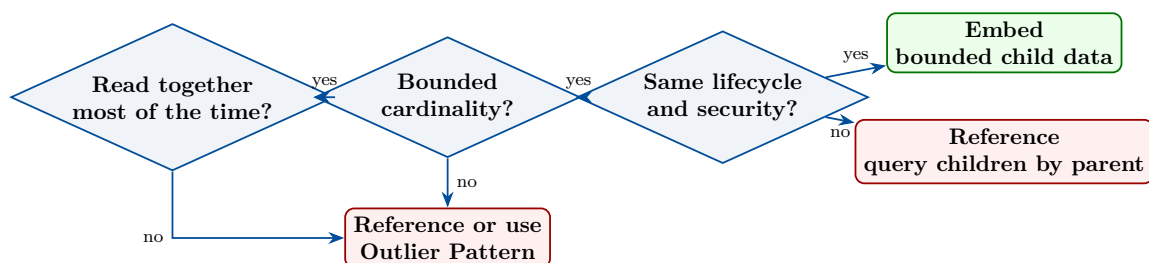


Figure 3.1: Embedding versus referencing decision tree for bounded relationships, shared lifecycle, and dominant read locality.

3.2.2 1:1 Relationships

For a 1:1 relationship, embedding is usually correct when the child is simply a component of the parent. An employee profile may embed emergency contact preferences or workstation assignment metadata if they are read with the employee and remain small. A payroll profile, however, may require separate access controls, encryption policy, audit handling, and retention rules; those concerns justify a reference even with 1:1 cardinality.

Tip

For 1:1 relationships, look beyond count. Separate the document when the child has a different compliance boundary, owner, retention period, or update workflow.

3.2.3 1:N Few Relationships

A 1:N few relationship is the document model's sweet spot. User addresses, employee skills, product images, and current role assignments are often embedded because the array is small and naturally read with the parent. The phrase *few* must be enforced by product rules or validation. If a user may eventually have millions of activity events, the relationship is not 1:N few even if the first week has only ten events.

Common Pitfall

Assuming current data volume is the design limit. A development sample with five children does not prove the relationship is bounded. Bound it with domain policy, validation, or a different model.

3.2.4 1:N Many Relationships

A 1:N many relationship should usually use parent referencing: each child document stores the parent's identifier and indexes it. A user activity event stores `userId`; a support ticket stores `accountId`; an order line may store `orderId` if line counts can be very large or independently processed. Parent referencing lets the child collection grow, paginate, expire, shard, and archive independently.

Definition

Parent referencing stores the parent's identifier in each child document, enabling queries such as `find({ userId: ... })` without growing the parent document.

3.2.5 N:M Relationships

N:M relationships need an edge collection, similar to a relational junction table. A training platform may store `employee_course_enrollments` with `employeeId`, `courseId`, `status`, `enrollment date`, `completion date`, and stable duplicated fields such as course title. Create indexes for both traversal directions.

Tip

For N:M edges, design the edge document as a first-class entity. It often has its own lifecycle, status, audit fields, and uniqueness rule, not merely two identifiers.

3.3 Tuesday: Document Anti-Patterns

3.3.1 Unbounded Arrays

An unbounded array grows without a strict maximum count or size. It may begin harmlessly as `activityLog: []`, `messages: []`, or `loginHistory: []`. Over time, the parent document becomes large, updates move more bytes, indexes on array fields become more expensive, and a single hot parent can serialize a high-volume write path.

Definition

An **unbounded array** is an array whose maximum number of elements is not controlled by domain rules, validation, bucketing, archival, or an overflow collection.

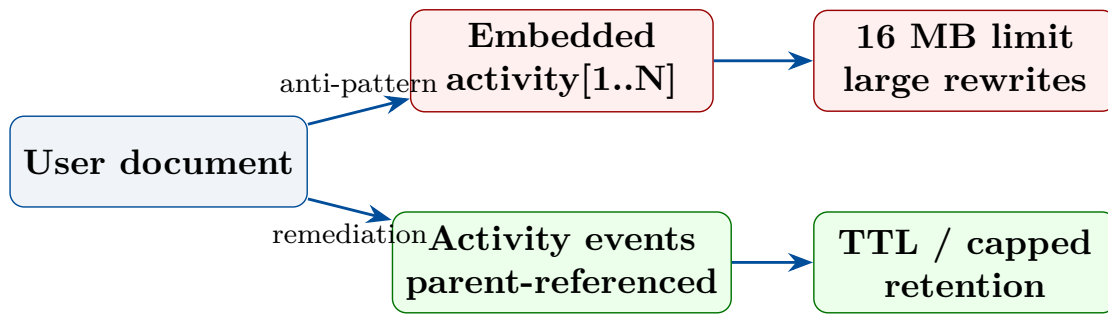


Figure 3.2: Unbounded arrays turn a convenient embedded design into a size-limit and write-amplification failure.

Common Pitfall

Embedding history forever. Logs, clicks, messages, audit trails, and telemetry are rarely safe as growing arrays inside an account or user document.

3.3.2 Bloated Documents and the 16 MB Limit

MongoDB stores documents as BSON and enforces a maximum document size of 16 MB [20]. The limit protects the server from documents that would be too expensive to move, replicate, and keep in memory. A bloated document may also be a performance problem long before it reaches the hard limit because every read and write touches unnecessary bytes.

Note

The 16 MB limit is not a target. A healthy hot document is usually much smaller and contains only data needed together by a known access path.

Tip

Track approximate document size in load tests with `Object.bsonsize(doc)` in `mongosh`, collection statistics, and representative production samples. Alert on growth trends, not just hard failures.

3.3.3 Massive Collections Without Shape Strategy

A massive collection is not automatically an anti-pattern. MongoDB can handle very large collections when indexes, shard keys, retention, and query filters are designed deliberately. The anti-pattern is a collection that grows without lifecycle policy, selective indexes, archive path, or query boundaries. User activity, observability, and event streams require time and tenant keys, retention windows, and operational ownership.

Common Pitfall

One giant collection with no lifecycle. If data never expires, never archives, and every query scans by a low-selectivity field, collection size will eventually dominate cost and incident response.

3.4 Wednesday: JSON Schema Validation

3.4.1 Validation Goals

MongoDB JSON Schema validation lets the database reject documents that violate structural rules. It is not a replacement for application validation, but it is a strong boundary for shared collections, migrations, and critical domains. A validator can require fields, restrict BSON types, enforce enums, validate nested objects, and reject unknown or oversized patterns when combined with application checks [19].

Definition

JSON Schema validation in MongoDB uses the `$jsonSchema` operator inside a collection validator to enforce document shape and selected field rules at insert and update time.

Note

MongoDB validators run on writes. They protect the collection boundary, but they do not automatically repair existing invalid documents unless validation level and migration logic address them.

3.4.2 Structural Constraints, Types, and Enums

A validator should express durable invariants: required identifiers, tenant ownership, lifecycle state, date fields, bounded arrays, and domain enums. In HR systems, examples include employee status, worker type, department identifier, and effective dates. Use `bsonType` for MongoDB-specific types such as `objectId`, `date`, `decimal`, and `int`.

Tip

Use validation for invariants that must never be bypassed, and use application code for user-friendly messages, cross-collection existence checks, and workflow-specific rules.

Common Pitfall

Encoding every business workflow in one validator. Overly complex validators become hard to migrate. Keep validators focused on stable structural boundaries and domain states.

3.5 Thursday Hands-On Project: Enterprise HR Validator

3.5.1 Scenario

The HR data platform stores employee master records consumed by payroll, benefits, identity governance, and analytics. Invalid documents have caused downstream failures: missing employee identifiers, free-form employment status values, arrays of certifications with no limit, and profile documents that accidentally include large audit histories. The goal is to create a collection that enforces required fields, types, enums, bounded arrays, and size guardrails.

```
1 use hr_master;
2 db.employees.drop();
3
4 db.createCollection("employees", {
5   validator: {
6     $jsonSchema: {
7       bsonType: "object",
```

```

8      required: ["employeeId", "tenantId", "status", "workerType", "legalName",
"departmentId", "hireDate", "schemaVersion"],
9      additionalProperties: true,
10     properties: {
11         employeeId: {
12             bsonType: "string",
13             pattern: "^EMP-[0-9]{6}$",
14             description: "employeeId must match EMP-000000"
15         },
16         tenantId: { bsonType: "string" },
17         status: { enum: ["active", "leave", "terminated"] },
18         workerType: { enum: ["employee", "contractor", "intern"] },
19         legalName: {
20             bsonType: "object",
21             required: ["given", "family"],
22             properties: {
23                 given: { bsonType: "string", maxLength: 80 },
24                 family: { bsonType: "string", maxLength: 80 },
25                 preferred: { bsonType: "string", maxLength: 80 }
26             }
27         },
28         departmentId: { bsonType: "string" },
29         managerId: { bsonType: ["string", "null"] },
30         hireDate: { bsonType: "date" },
31         terminationDate: { bsonType: ["date", "null"] },
32         schemaVersion: { bsonType: "int", minimum: 1, maximum: 3 },
33         certifications: {
34             bsonType: "array",
35             maxItems: 25,
36             items: {
37                 bsonType: "object",
38                 required: ["code", "issuedAt"],
39                 properties: {
40                     code: { bsonType: "string", maxLength: 40 },
41                     issuedAt: { bsonType: "date" },
42                     expiresAt: { bsonType: ["date", "null"] }
43                 }
44             }
45         },
46         profileSummary: { bsonType: "string", maxLength: 2000 },
47         activityRef: { bsonType: ["objectId", "null"] }
48     }
49 },
50 },
51 validationLevel: "strict",
52 validationAction: "error"
53 });
54
55 db.employees.createIndex({ tenantId: 1, employeeId: 1 }, { unique: true });
56 db.employees.createIndex({ tenantId: 1, departmentId: 1, status: 1 });

```

Listing 3.1: mongosh collection creation with an enterprise HR \$jsonSchema validator.

The validator bounds the embedded certification list and profile summary. It does not embed audit history or activity logs. Those high-cardinality records belong in separate collections referenced by `employeeId` or `activityRef`.

```

1 from datetime import datetime, timezone
2 from pymongo import MongoClient
3 from pymongo.errors import WriteError
4
5 client = MongoClient("mongodb://127.0.0.1:27017")
6 db = client.hr_master

```

```

7 employees = db.employees
8
9 valid_employee = {
10     "employeeId": "EMP-000123",
11     "tenantId": "tenant-acme",
12     "status": "active",
13     "workerType": "employee",
14     "legalName": {"given": "Avery", "family": "Nguyen", "preferred": "Avery"},
15     "departmentId": "DEPT-DATA",
16     "managerId": None,
17     "hireDate": datetime(2024, 5, 1, tzinfo=timezone.utc),
18     "terminationDate": None,
19     "schemaVersion": 1,
20     "certifications": [
21         {"code": "MONGO-DE", "issuedAt": datetime(2026, 7, 31, tzinfo=timezone.
22         utc), "expiresAt": None}
23     ],
24     "profileSummary": "Senior data engineer supporting MongoDB platforms.",
25     "activityRef": None,
26 }
27
28 invalid_employee = {
29     "employeeId": "123",
30     "tenantId": "tenant-acme",
31     "status": "active-ish",
32     "workerType": "employee",
33     "legalName": {"given": "Jordan"},
34     "departmentId": "DEPT-DATA",
35     "hireDate": "2026-07-31",
36     "schemaVersion": 99,
37     "certifications": [{"code": f"CERT-{i}", "issuedAt": datetime.now(timezone.
38     utc)} for i in range(30)],
39 }
40
41 employees.delete_many({"tenantId": "tenant-acme", "employeeId": {"$in": ["EMP
42     -000123", "123"]}})
43 employees.insert_one(valid_employee)
44 print("valid employee inserted")
45
46 try:
47     employees.insert_one(invalid_employee)
48 except WriteError as exc:
49     print("invalid employee rejected")
50     print(exc.details.get("errmsg", str(exc)))

```

Listing 3.2: PyMongo validation test that inserts a valid HR document and handles validation errors.

Note

The script expects the collection to already exist with the validator. In production, deploy validator changes through versioned migration scripts and test them against representative documents before enforcing `validationAction: "error"`.

3.6 Friday Use Case and Architecture: Remediating 16 MB Activity-Log Failures

3.6.1 Scenario and Root Cause

A legacy customer portal embeds user activity inside each user document: `{ _id, userId, profile, activityLog: [ ... millions of events ... ] }`. As the highest-traffic users accumulate years of login, click, export, and security events, updates begin failing with `BSOObj size exceeds 16MB`. The design also makes retention impossible because old events cannot be deleted without rewriting the user document.

The remediation separates identity from activity. User profile data remains in `users`. New events are written to `user_activity_events` with parent references, indexed by `tenantId`, `userId`, and event time. A capped collection keeps the newest operational firehose for fast diagnostics, while a normal collection or archival pipeline stores records requiring audit retention.

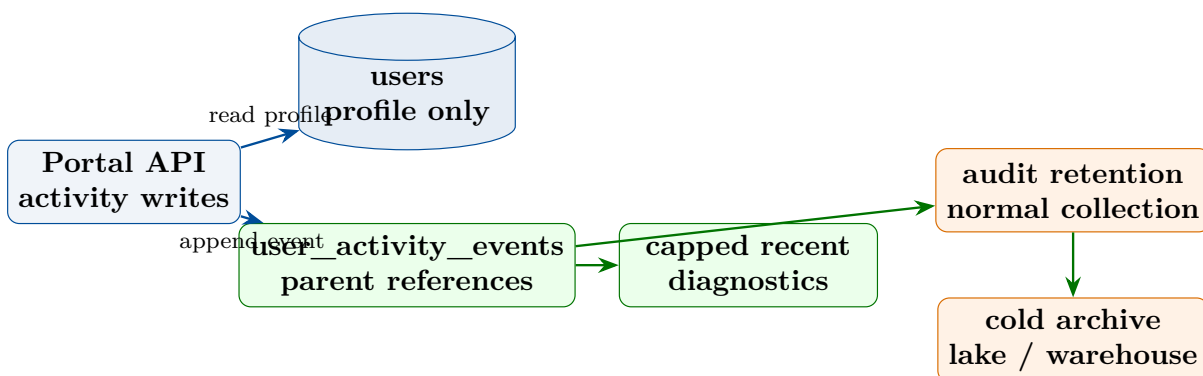


Figure 3.3: Parent-referenced activity events and capped recent diagnostics replace an unbounded embedded activity array.

Tip

Use capped collections for bounded recent streams where automatic overwrite is acceptable. Do not use them as the only store for records that require legal, security, or customer audit retention [16].

Common Pitfall

Replacing one unbounded array with one unbounded retention problem. The activity collection still needs indexes, time windows, archival, and deletion or tiering rules.

3.7 Interview Q&A

3.7.1 Concepts and Trade-offs

1. **Q: How do you choose embedding versus referencing?**

A: Embed data that is read together, bounded, and shares lifecycle. Reference data that grows independently, updates independently, has separate security, or needs its own query and retention path.

2. **Q: Why are unbounded arrays dangerous in MongoDB?**

A: They increase document size, amplify updates, can create multikey index overhead, and eventually risk the 16 MB BSON document limit.

3. **Q: When would you use `$lookup`?**

A: Use it for targeted joins when references are the right model and indexes support the join predicates. Do not use it to excuse a relationship model that ignores cardinality and access patterns.

4. **Q: How do you model N:M relationships?**

A: Use an edge collection with identifiers for both sides, a uniqueness rule, traversal indexes in both directions, and any relationship attributes such as status or effective dates.

5. **Q: What does JSON Schema validation protect?**

A: It protects structural invariants at the database boundary: required fields, types, enums, nested shapes, array bounds, and selected value ranges.

6. **Q: Can MongoDB enforce foreign keys like a relational database?**

A: Not as declarative foreign-key constraints. Applications usually enforce references, sometimes with transactions, change streams, or reconciliation jobs for bounded workflows.

7. **Q: What should you do after seeing `BSONObj` size exceeds 16MB?**

A: Identify the growing field, stop new growth, move high-cardinality data to a referenced collection, migrate existing history, add validation or limits, and add retention controls.

3.8 Further Reading

Start with MongoDB data modeling, document limit, `$lookup`, JSON Schema validation, and capped collection documentation [17, 20, 14, 19, 16]. For adjacent platform comparison, review DynamoDB single-table design and Cosmos DB partitioning guidance [1, 8]. For schema patterns that complement this chapter, revisit the Bucket, Outlier, Extended Reference, and Schema Versioning Patterns from Chapter 2.

Key Takeaways

- Embedding is ideal for bounded, co-read, same-lifecycle data; referencing is safer for large, independently queried, independently updated, or separately retained data.
- 1:1 and 1:N few relationships often embed, while 1:N many and N:M relationships usually require parent references or edge collections.
- Unbounded arrays and bloated documents are production anti-patterns because they increase write cost and can reach MongoDB's 16 MB BSON document limit.
- Massive collections are manageable only when indexes, retention, shard keys, archival, and query boundaries are explicit.
- JSON Schema validation enforces durable collection invariants, but application code still handles friendly errors and cross-collection workflow rules.
- Capped collections are useful for bounded recent streams, not for the sole copy of regulated audit history.

3.9 Exercises

1. **(Conceptual)** Classify employee profile, payroll profile, emergency contacts, certifications, and audit events as embedded or referenced. Justify each decision with lifecycle and cardinality.
2. **(Conceptual)** Explain why a current array length of five does not prove a relationship is safe to embed forever.
3. **(Hands-on)** Write a `$jsonSchema` validator for a department collection with required `tenantId`, `departmentId`, `name`, and enum `status`.
4. **(Hands-on)** Create indexes for an N:M `employee_course_enrollments` edge collection supporting employee transcript and course roster queries.
5. **(Design)** Re-model a social-media comments array that can reach one million comments per post. Include pagination, indexes, and retention.
6. **(Design)** Propose a migration plan for moving embedded user activity arrays into a parent-referenced collection without downtime.
7. **(Performance)** Estimate the risk of a 2 KB activity event appended 10,000 times to one user document. Compare that with 10,000 referenced event documents.
8. **(Interview)** Write a concise answer to: *Does MongoDB support joins, and does that mean we should normalize every relationship?*

3.10 Capstone Project: Remediate a 16 MB Activity-Log Failure

Capstone Project

Mission: remediate a production activity-log design that embeds unbounded user events and fails with `BSONObj size exceeds 16MB`. You will design parent-referenced event storage, create a capped recent diagnostics collection, enforce JSON Schema validation, migrate sample data, and prove that the new profile document remains bounded.

Estimated time: 5–7 hours, depending on migration sample size and validation depth.

What you will practice:

- Diagnosing document growth from embedded high-cardinality arrays.
- Creating normal and capped collections for different retention purposes.
- Applying `$jsonSchema` validators to user profile and activity event collections.
- Writing migration code that moves embedded activity into parent-referenced documents.
- Validating document sizes, counts, indexes, and failure-mode remediation evidence.

Scenario

A SaaS portal stores every user event inside `users.activityLog`. Enterprise tenants with automated exports generate hundreds of thousands of events per user. Profile updates now fail for heavy users, incident responders cannot query recent activity efficiently, and compliance requires one year of selected security events. The architecture must keep user profiles small, retain recent operational events, preserve required audit history, and reject any future attempt to reintroduce unbounded embedded activity.

Requirements

- Keep **users** limited to profile, account status, stable preferences, and small bounded subsets only.
- Store each activity event in `user_activity_events` with `tenantId`, `userId`, `eventId`, `eventType`, `occurredAt`, and payload metadata.
- Maintain a capped `recent_activity_stream` collection for recent diagnostics where over-write is acceptable.
- Add JSON Schema validation that rejects embedded `activityLog` on user profiles and enforces required event fields.
- Create indexes for user timeline reads, tenant security investigations, and event deduplication.
- Provide a migration path from embedded arrays to referenced event documents with idempotent writes.
- Define archival and retention rules for one-year audit events and shorter operational diagnostics.

Reference Architecture

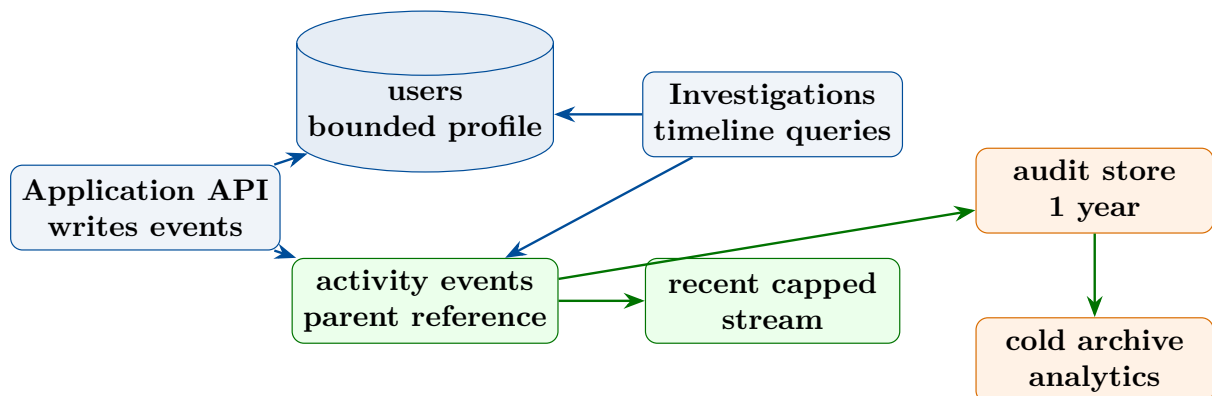


Figure 3.4: Capstone reference architecture with bounded user profiles, parent-referenced events, capped diagnostics, and audit archival.

Implementation Steps

1. **Create bounded collections and indexes.** The setup below prevents embedded activity on the user profile and creates separate event stores.

```

1 use activity_remediation;
2 db.users.drop();
3 db.user_activity_events.drop();
4 db.recent_activity_stream.drop();
5
6 db.createCollection("users", {
7   validator: {
8     $jsonSchema: {
9       bsonType: "object",
10      required: ["tenantId", "userId", "email", "status", "schemaVersion"],
11      properties: {
12        tenantId: { bsonType: "string" },

```

```

13     userId: { bsonType: "string" },
14     email: { bsonType: "string" },
15     status: { enum: ["active", "locked", "disabled"] },
16     schemaVersion: { bsonType: "int", minimum: 2 },
17     activityLog: { bsonType: "undefined", description: "activityLog
    must not be embedded" }
18   }
19 }
20 },
21 validationAction: "error"
22 });
23
24 db.createCollection("user_activity_events", {
25   validator: {
26     $jsonSchema: {
27       bsonType: "object",
28       required: ["tenantId", "userId", "eventId", "eventType", "occurredAt"
29     ],
29     properties: {
30       tenantId: { bsonType: "string" },
31       userId: { bsonType: "string" },
32       eventId: { bsonType: "string" },
33       eventType: { enum: ["login", "logout", "export", "permission_change
34       ", "api_call"] },
35       occurredAt: { bsonType: "date" },
36       payload: { bsonType: ["object", "null"] }
37     }
38   },
39   validationAction: "error"
40 });
41
42 db.createCollection("recent_activity_stream", { capped: true, size:
43   104857600, max: 500000 });
44 db.users.createIndex({ tenantId: 1, userId: 1 }, { unique: true });
45 db.user_activity_events.createIndex({ tenantId: 1, userId: 1, occurredAt:
46   -1 });
47 db.user_activity_events.createIndex({ tenantId: 1, eventType: 1, occurredAt:
48   -1 });
49 db.user_activity_events.createIndex({ tenantId: 1, eventId: 1 }, { unique:
50   true });

```

Listing 3.3: mongosh setup for bounded users, referenced activity events, and capped diagnostics.

2. **Migrate embedded arrays idempotently.** The script writes each embedded event as a child document, mirrors it into the capped stream, upgrades the user schema, and removes the embedded array.

```

1 from datetime import datetime, timezone
2 from pymongo import MongoClient, UpdateOne
3 from pymongo.errors import BulkWriteError
4
5 client = MongoClient("mongodb://127.0.0.1:27017")
6 db = client.activity_remediation
7 users = db.legacy_users
8 profiles = db.users
9 events = db.user_activity_events
10 recent = db.recent_activity_stream
11
12 sample_user = {
13     "tenantId": "tenant-acme",
14     "userId": "user-001",
15     "email": "user001@example.com",

```

```

16     "status": "active",
17     "schemaVersion": 1,
18     "activityLog": [
19         {"eventId": "evt-1", "eventType": "login", "occurredAt": datetime
20         (2026, 7, 31, 10, 0, tzinfo=timezone.utc), "payload": {"ip": "10.0.0.5"
21         }},
22         {"eventId": "evt-2", "eventType": "export", "occurredAt": datetime
23         (2026, 7, 31, 10, 5, tzinfo=timezone.utc), "payload": {"rows": 1200}},
24     ],
25 }
26 users.replace_one({"tenantId": sample_user["tenantId"], "userId":
27     sample_user["userId"]}, sample_user, upsert=True)
28
29 for user in users.find({"activityLog.0": {"$exists": True}}):
30     tenant_id = user["tenantId"]
31     user_id = user["userId"]
32     event_ops = []
33     recent_docs = []
34     for event in user.get("activityLog", []):
35         child = {
36             "tenantId": tenant_id,
37             "userId": user_id,
38             "eventId": event["eventId"],
39             "eventType": event["eventType"],
40             "occurredAt": event["occurredAt"],
41             "payload": event.get("payload"),
42         }
43         event_ops.append(UpdateOne(
44             {"tenantId": tenant_id, "eventId": event["eventId"]},
45             {"$setOnInsert": child},
46             upsert=True,
47         ))
48         recent_docs.append(child)
49
50 if event_ops:
51     try:
52         events.bulk_write(event_ops, ordered=False)
53     except BulkWriteError as exc:
54         print("duplicate or validation issue", exc.details.get("
55 writeErrors", [])[0])
56
57 if recent_docs:
58     recent.insert_many(recent_docs, ordered=False)
59
60 profiles.update_one(
61     {"tenantId": tenant_id, "userId": user_id},
62     {"$set": {
63         "tenantId": tenant_id,
64         "userId": user_id,
65         "email": user["email"],
66         "status": user["status"],
67         "schemaVersion": 2,
68     }},
69     upsert=True,
70 )
71 users.update_one({"_id": user["_id"]}, {"$unset": {"activityLog": ""},
72 "$set": {"schemaVersion": 2}})
73
74 print({
75     "profiles": profiles.count_documents({}),
76     "events": events.count_documents({}),
77     "recent": recent.count_documents({}),

```

```
71 })
```

Listing 3.4: PyMongo migration from embedded activity arrays to parent-referenced events.

3. **Validate the fix.** Check that user profiles reject future embedded activity, event queries use indexes, and profile document size remains bounded.

```
1 use activity_remediation;
2 try {
3   db.users.insertOne({
4     tenantId: "tenant-acme",
5     userId: "bad-user",
6     email: "bad@example.com",
7     status: "active",
8     schemaVersion: 2,
9     activityLog: [{ eventId: "should-not-embed" }]
10  });
11 } catch (e) {
12   print("embedded activity rejected: " + e.codeName);
13 }
14
15 printjson(db.user_activity_events.find({
16   tenantId: "tenant-acme",
17   userId: "user-001"
18 }).sort({ occurredAt: -1 }).explain("executionStats"));
19
20 const profile = db.users.findOne({ tenantId: "tenant-acme", userId: "user-001" });
21 print("profile BSON bytes: " + Object.bsonsize(profile));
```

Listing 3.5: mongosh validation checks for the remediated activity-log design.

Deliverables

- A before-and-after schema design note explaining why embedded activity failed and why parent referencing fixes the growth mode.
- The `mongosh` setup script for validators, indexes, normal event collection, and capped recent stream.
- The PyMongo migration script with idempotent event writes and evidence that embedded arrays are removed.
- A validation report showing profile document size, event counts, rejected embedded activity, and indexed timeline query execution statistics.
- A retention plan identifying which events remain in MongoDB, which enter capped diagnostics, and which move to audit archive.

Acceptance Criteria

- New user profile documents cannot include `activityLog`.
- Activity events are stored as parent-referenced child documents with indexes for user timelines and tenant investigations.
- The capped collection has explicit byte and document limits and is not the only audit store.
- Migration is idempotent by `tenantId` and `eventId`.
- The design prevents future 16 MB profile failures caused by activity growth.

- Validation errors are demonstrated with an intentionally invalid write.
- The architecture includes retention and archival rules for compliance-sensitive activity.

Stretch Goals

- Add a TTL index for non-audit activity events after archive confirmation.
- Add a change stream that copies selected event types into the audit store.
- Add a reconciliation job that compares legacy array counts with migrated event counts before deleting legacy fields.
- Model a sharded version using a tenant-aware shard key and evaluate targeted user timeline reads.
- Create a dashboard query that summarizes activity by event type without reading user profiles.
- Add schema versioning so readers can tolerate profiles during the migration window.
- Measure average object size before and after migration and estimate storage avoided for 100,000 heavy users.

Relational to Document Migration Strategy

Learning Objectives

- Analyze normalized relational schemas and identify 3NF table groups that should collapse into bounded MongoDB documents.
- Use MongoDB Relational Migrator concepts: source connection, destination cluster, mapping model, transformation rules, initial sync, and continuous CDC.
- Design zero-downtime cutovers using dual-writes, shadow reads, CDC replication pipelines, reconciliation, and instant rollback.
- Map a 12-table PostgreSQL e-commerce schema into three MongoDB collections and verify initial sync plus CDC with PyMongo.
- Plan a 15 TB PostgreSQL-to-Atlas migration with rollback guarantees, lossless replication, query parity, and operational checkpoints.
- Close Part I by integrating schema patterns, relationship modeling, JSON Schema validation, Relational Migrator CDC, and performance benchmarking.

Cross-Cloud Equivalence

Database migration services use different product names, but production teams still solve the same problems: schema conversion, full load, continuous change capture, cutover safety, and rollback. MongoDB Relational Migrator focuses on moving relational workloads into document models, while cloud-native migration services often preserve a relational target unless paired with a schema redesign step.

Capability	MongoDB Relational Migrator	AWS DMS + SCT	Azure Database Migration Service	GCP Database Migration Service
Schema conversion	Converts relational tables into MongoDB collections with embedded arrays, references, and transformation rules	SCT converts relational engines and recommends target schema changes; DMS moves data	Assesses and migrates database engines, usually relational-to-relational with compatibility guidance	Migrates source engines into Cloud SQL or AlloyDB with schema compatibility checks
Full-load + CDC	Performs initial sync and keeps MongoDB synchronized through CDC from supported relational sources	DMS supports full load plus ongoing replication tasks	Online migrations use continuous sync until cutover when supported	Continuous replication supports low-downtime migrations for supported sources
Dual-write / shadow-read cutover	Application pattern around the migrator: write both systems, compare reads, then promote Atlas	Application pattern around DMS tasks and target database validation	Application pattern around online migration and application routing	Application pattern around replication lag, validation, and cutover windows
Rollback strategy	Keep PostgreSQL authoritative until final promotion; after promotion, retain dual-writes or reverse sync window for instant rollback	Retain source as system of record; rollback by routing reads and writes back to source	Retain source and DNS/application routing until validation passes	Retain source and delay destructive changes until post-cutover confidence window

Microsoft Fabric and Snowflake often enter the same program as analytical ingestion targets rather than operational cutover targets. They can consume CDC streams or staged exports for validation, reconciliation, and analytics, but they do not replace the operational document-modeling decisions required for MongoDB Atlas [28, 2, 3, 9, 6, 31, 10].

4.1 Week 4 Roadmap: Migration as Document-Model Modernization

Relational-to-document migration is not a file copy. A successful migration preserves business meaning while changing physical shape. Third normal form tables that were optimized to prevent update anomalies often become embedded subdocuments, bounded arrays, or referenced collections optimized for application access paths. This week connects the modeling rules from Chapters 2 and 3 to a migration program that can run continuously, prove parity, and cut over without downtime.

Definition

Relational-to-document migration is the controlled redesign and movement of data from normalized tables into MongoDB collections whose document boundaries match access patterns, cardinality, lifecycle, and operational constraints.

Key Terms

3NF: third normal form, where non-key attributes depend on the key and repeating facts move to related tables. **CDC**: change data capture, the replication of committed source changes. **Initial sync**: bulk copy before ongoing replication catches up. **Dual-write**: writing source and target during a transition. **Shadow read**: reading the new system for comparison while serving users from the old system. **Cutover**: promoting the new system to serve production traffic.

4.2 Monday: Analyzing RDBMS Schemas

4.2.1 From Tables to Aggregate Boundaries

A normalized schema describes dependencies, not necessarily application locality. The migration engineer begins by reading foreign keys, uniqueness constraints, row counts, update rates, and top queries. Tables that are always loaded together, share lifecycle, and have bounded child counts are candidates for one MongoDB document. Tables that grow independently, support separate search, or require separate retention remain separate collections with references.

Definition

An **aggregate boundary** is the document or collection boundary around facts that should be read, validated, updated, and versioned together for a dominant use case.

Note

Do not translate every table into one collection. That produces a document database with relational pain: excessive \$lookup, fragmented reads, and no benefit from locality.

4.2.2 Finding 3NF Tables to Merge

Look for dependent tables whose primary key is also a foreign key to the parent, such as `customer_profile`, `customer_preferences`, and `customer_addresses`. Then check child cardinality. A customer with two addresses is a bounded embedded array; a customer with ten million events is a referenced event collection. The safe merge list usually includes profile extensions, current settings, small code tables copied as labels, and line items when order size is bounded by business rules.

Tip

Build a migration worksheet with columns for parent table, child table, cardinality maximum, read-together percentage, update owner, retention period, security boundary, proposed MongoDB shape, and rollback risk.

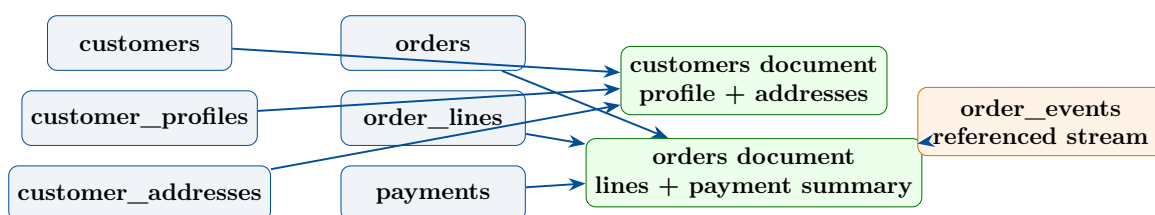


Figure 4.1: Normalized tables collapse into bounded documents, while unbounded event history remains referenced.

Common Pitfall

Embedding because a foreign key exists. A foreign key proves relationship, not boundedness. Migration design still needs volume, growth, access pattern, and lifecycle evidence.

4.2.3 Merge, Reference, or Duplicate

Migration decisions are rarely binary. A product catalog may embed current price display fields in order line items for historical correctness, reference the product collection for current product details, and store full product change history elsewhere. This controlled duplication is normal in document modeling. It must be documented as part of the migration contract so CDC transformations keep duplicated fields consistent where required.

4.3 Tuesday: MongoDB Relational Migrator

4.3.1 Mapping Model and Transformation Rules

MongoDB Relational Migrator connects to relational sources, inspects schemas, helps design a MongoDB mapping, performs initial sync, and can keep the target synchronized through CDC for supported databases [28]. The important design artifact is the mapping model: which tables form a collection, which columns become nested fields, which child rows become arrays, and which tables remain separate collections.

Definition

A **migration mapping** is an executable specification that turns source tables and rows into target MongoDB documents, including field renames, type conversions, embedded arrays, references, filters, and computed values.

Note

Transformation rules are part of the data contract. Store them with version control, review them like application code, and test them with representative source rows before running production sync.

4.3.2 CDC Setup

CDC reads committed database changes after the initial load and applies equivalent changes to the MongoDB target. For PostgreSQL this usually means enabling logical replication, creating an appropriate publication or replication slot, granting least-privilege access, and monitoring lag. CDC does not remove the need for reconciliation. It narrows the cutover window by keeping the target nearly current.

Tip

Track four CDC numbers during rehearsal: source commit position, target applied position, replication lag in seconds, failed event count, and reconciliation difference by collection. Promote only when all are inside the agreed threshold.

Common Pitfall

Treating CDC as backup. CDC is a migration and replication mechanism, not a recoverability strategy. Keep source backups, Atlas backups, point-in-time recovery, and tested rollback paths.

4.3.3 Type and Constraint Translation

Relational constraints do not move automatically into MongoDB. Primary keys often become `_id` or stable business keys. Foreign keys become embedded context, references, or validation

plus reconciliation jobs. Numeric precision maps to Decimal128 when money or exact quantities require it. Enums map to JSON Schema enum lists. Nullable columns require explicit decisions: omit absent optional fields, store `null`, or provide defaults.

4.4 Wednesday: Zero-Downtime Migration Patterns

4.4.1 Dual-Writes and Shadow Reads

Zero downtime requires the application to tolerate both systems during transition. Dual-writes send new changes to PostgreSQL and MongoDB. Shadow reads execute MongoDB reads in parallel, compare results, but still serve the user from PostgreSQL until confidence is high. The comparison layer should normalize harmless representation differences such as field order, decimal encoding, and timestamp precision.

Definition

A **shadow read** is a production read executed against the future target system for comparison and telemetry while the current system remains authoritative for user-facing responses.

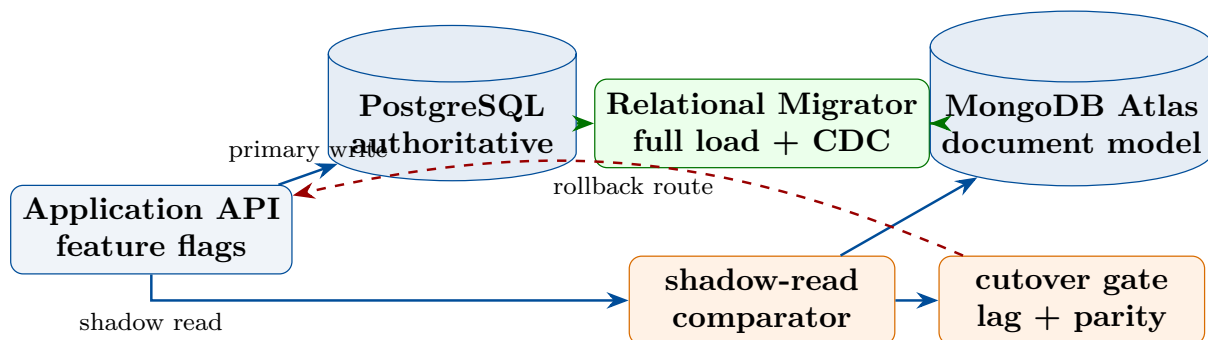


Figure 4.2: Dual-write and CDC cutover pipeline with shadow-read comparison and rollback routing.

Note

A zero-downtime migration is an operational state machine: prepare, full-load, catch up, shadow compare, limited canary, promote reads, promote writes, observe, and only then retire the old path.

4.4.2 CDC Replication Pipelines

The CDC pipeline must be observable. Each source transaction should either apply successfully to MongoDB or enter a dead-letter process with enough context to repair it. Common failures include type overflow, missing parent rows during transformation, duplicate keys caused by earlier test loads, and array growth that violates the new document boundary.

Tip

Prefer idempotent target writes during migration. Use deterministic identifiers derived from source primary keys so retries update or upsert the same document instead of creating duplicates.

4.4.3 Rollback and Data-Loss Boundaries

Instant rollback means traffic can be routed back to PostgreSQL without losing acknowledged writes. Before final promotion, this is simple because PostgreSQL remains authoritative. After promotion, rollback requires either a continued dual-write window, reverse replication, or a strict rule that PostgreSQL remains write-primary until the rollback window expires. The design must state exactly which system owns writes at every minute.

Common Pitfall

Cutting over reads and writes at the same time without a rollback writer. If MongoDB accepts unique writes after PostgreSQL stops receiving them, instant rollback is impossible unless those writes are replicated back or replayable from a durable log.

4.5 Thursday Hands-On Project: E-Commerce Relational Migrator Lab

4.5.1 Scenario

A PostgreSQL e-commerce database contains twelve normalized tables: `customers`, `customer_profiles`, `customer_addresses`, `products`, `product_attributes`, `product_categories`, `orders`, `order_lines`, `payments`, `shipments`, `inventory_items`, and `inventory_movements`. The migration target has three MongoDB collections: `customers`, `orders`, and `catalog`. Inventory movement history remains referenced inside `catalog.inventoryRefs` because it can grow independently.

```

1 project: ecommerce-modernization
2 source:
3   type: postgresql
4   host: pg-source.internal
5   database: ecommerce
6   logicalReplication: true
7   publication: migrator_pub
8   slot: ecommerce_to_atlas
9
10 target:
11   type: mongodb-atlas
12   database: ecommerce_doc
13   writeMode: upsert
14   idStrategy: deterministic-source-key
15
16 collections:
17   customers:
18     rootTable: customers
19     id: customer_id
20     fields:
21       customerId: customers.customer_id
22       email: customers.email
23       status: customers.status
24       createdAt: customers.created_at
25     profile:
26       from: customer_profiles
27       cardinality: one
28       fields: [first_name, last_name, loyalty_tier, birth_date]
29     addresses:
30       from: customer_addresses
31       cardinality: bounded-many
32       maxItems: 10
33       fields: [address_id, type, line1, city, region, postal_code, country]
34   orders:

```

```

35     rootTable: orders
36     id: order_id
37     fields:
38         orderId: orders.order_id
39         customerId: orders.customer_id
40         status: orders.status
41         placedAt: orders.placed_at
42         totals: [subtotal, tax, shipping, grand_total]
43         lines:
44             from: order_lines
45             cardinality: bounded-many
46             maxItems: 200
47             fields: [line_id, product_id, sku, name_snapshot, quantity, unit_price]
48         paymentSummary:
49             from: payments
50             cardinality: one
51             fields: [payment_id, method, status, authorized_at]
52         shipmentSummary:
53             from: shipments
54             cardinality: one
55             fields: [shipment_id, carrier, tracking_number, shipped_at]
56 catalog:
57     rootTable: products
58     id: product_id
59     fields:
60         productId: products.product_id
61         sku: products.sku
62         name: products.name
63         status: products.status
64         categories:
65             from: product_categories
66             cardinality: bounded-many
67             fields: [category_id, category_name]
68         attributes:
69             from: product_attributes
70             cardinality: attribute-pattern
71             keyField: attribute_name
72             valueField: attribute_value
73         inventoryRefs:
74             from: inventory_items
75             cardinality: references
76             fields: [warehouse_id, available_quantity]
77         movementCollection: inventory_movements
78 cdc:
79     mode: continuous
80     applyDeletes: soft-delete
81     lagAlertSeconds: 30
82     deadLetterCollection: migration_dead_letters

```

Listing 4.1: Relational Migrator-style mapping from twelve PostgreSQL tables into three MongoDB collections.

The mapping intentionally embeds order lines because order size is bounded by business policy and order reads require line details. It embeds product attributes using the Attribute Pattern introduced in Chapter 2. It does not embed inventory movements because warehouse adjustments can grow forever and need time-based reconciliation.

```

1 from datetime import datetime, timezone
2 from time import sleep
3 from pymongo import MongoClient
4
5 client = MongoClient("mongodb://127.0.0.1:27017")
6 db = client.ecommerce_doc

```

```

7
8 expected_minimums = {
9     "customers": 1000,
10    "orders": 5000,
11    "catalog": 500,
12 }
13
14 for collection_name, minimum in expected_minimums.items():
15     actual = db[collection_name].count_documents({})
16     print(f"{collection_name}: {actual}")
17     assert actual >= minimum, f"{collection_name} below expected initial sync
    count"
18
19 sample_order = db.orders.find_one({"lines.0": {"$exists": True}})
20 assert sample_order is not None, "orders must embed order lines"
21 assert "paymentSummary" in sample_order, "orders must include payment summary"
22 assert "shipmentSummary" in sample_order, "orders must include shipment summary
    "
23
24 cdc_probe_id = "cdc-probe-1001"
25 db.customers.update_one(
26     {"customerId": cdc_probe_id},
27     {"$set": {
28         "customerId": cdc_probe_id,
29         "email": "cdc-probe@example.com",
30         "status": "active",
31         "createdAt": datetime.now(timezone.utc),
32         "profile": {"first_name": "CDC", "last_name": "Probe", "loyalty_tier":
            "gold"},
33     }},
34     upsert=True,
35 )
36
37 sleep(2)
38 probe = db.customers.find_one({"customerId": cdc_probe_id})
39 assert probe and probe["profile"]["loyalty_tier"] == "gold"
40
41 lag_doc = db.migration_status.find_one({"pipeline": "ecommerce_to_atlas"}) or
    {}
42 print({
43     "cdcProbeFound": bool(probe),
44     "lastApplied": lag_doc.get("lastApplied"),
45     "lagSeconds": lag_doc.get("lagSeconds"),
46 })

```

Listing 4.2: PyMongo verification script for initial sync counts, document shape, and CDC propagation.

Note

In a real lab, the CDC probe should be written to PostgreSQL and observed in MongoDB. This compact script demonstrates the target-side assertions to automate after Relational Migrator applies the source change.

4.6 Friday Use Case: Zero-Downtime 15 TB PostgreSQL Migration

4.6.1 Scenario and Requirements

A retail platform runs a 15 TB PostgreSQL monolith with order, catalog, customer, inventory, and fulfillment data. Peak writes cannot stop, weekend maintenance windows are too short for a bulk export, and the business requires instant rollback with zero acknowledged-write loss. The migration target is MongoDB Atlas with document models optimized for order lookups, customer account pages, and catalog search.

The migration plan keeps PostgreSQL authoritative through initial load, CDC catch-up, shadow reads, and read canaries. Writes remain dual-written for a defined rollback window after MongoDB begins serving reads. Only when reconciliation, SLOs, and incident drills pass does the team promote MongoDB as write-primary and retire the rollback path.

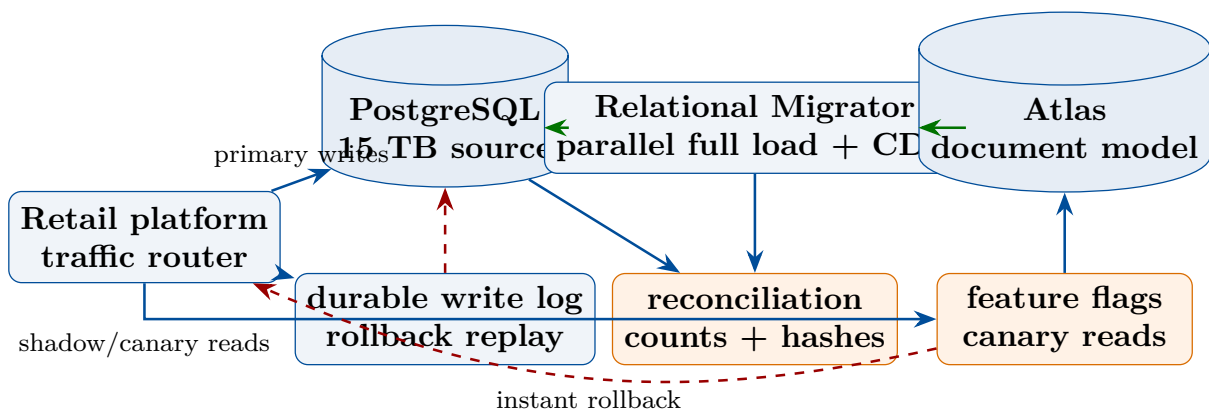


Figure 4.3: Zero-downtime 15 TB migration architecture with CDC, reconciliation, canary reads, durable write log, and rollback routing.

Tip

For very large migrations, rehearse with production-like partitions: largest tenant, hottest order ranges, widest product attributes, and oldest historical rows. A small average dataset hides tail risks.

Common Pitfall

Promoting before rollback is tested. A rollback plan that has not been executed under realistic traffic is documentation, not a control.

4.6.2 Operational Runbook

The runbook begins with schema freeze for incompatible relational changes, Atlas capacity validation, index build completion, and source logical replication health checks. Initial sync runs in parallel by table groups. CDC catches up while reconciliation compares row counts, document counts, sampled checksums, high-value orders, and application query results. Shadow reads run for at least one full business cycle. Canary reads expand by tenant or region. During the rollback window, PostgreSQL receives every acknowledged write directly or through a durable replay log. The final promotion occurs only after lag, parity, error budgets, and rollback drills pass.

4.7 Interview Q&A

4.7.1 Migration Design and Operations

1. **Q: Why should a relational-to-MongoDB migration not map one table to one collection?**
A: One-to-one mapping preserves relational fragmentation and usually forces excessive joins. The target model should follow access patterns, cardinality, lifecycle, and document-size limits.
2. **Q: Which 3NF tables are good candidates for embedding?**
A: Tables that are read with the parent, have bounded cardinality, share lifecycle and security, and do not need independent retention are good embedding candidates.
3. **Q: What does CDC solve during migration?**
A: CDC keeps the target current after the initial load so the final cutover window is short. It does not replace validation, backups, or rollback design.
4. **Q: How do dual-writes differ from CDC?**
A: CDC replicates committed source changes after the fact. Dual-writes make the application write both systems during transition, often to protect rollback after target reads or writes begin.
5. **Q: What is a shadow read?**
A: A read against the target system whose result is compared to the source while the user still receives the source response.
6. **Q: How do you guarantee zero data loss in rollback?**
A: Define a single write authority or maintain durable dual-writes/replay so every acknowledged write exists in the rollback target before traffic can return to it.
7. **Q: What metrics decide cutover readiness?**
A: CDC lag, failed event count, reconciliation differences, query parity, p95/p99 latency, error rate, index readiness, and successful rollback drill results.
8. **Q: How should relational constraints become MongoDB safeguards?**
A: Use deterministic identifiers, unique indexes, JSON Schema validation, application checks, transactions for bounded workflows, and reconciliation jobs for cross-collection integrity.

4.8 Further Reading

Start with MongoDB Relational Migrator, data modeling, schema patterns, JSON Schema validation, and Atlas documentation [28, 17, 13, 19]. For platform comparison, review AWS DMS and SCT, Azure Database Migration Service, and Google Database Migration Service [2, 3, 9, 6]. For source CDC planning, review PostgreSQL logical replication and write-ahead logging concepts [29, 30].

Key Takeaways

- Migration is a document-modeling exercise before it is a data-copy exercise.
- 3NF tables collapse safely only when cardinality is bounded and lifecycle, security, and access patterns align.

- MongoDB Relational Migrator mappings should be versioned, tested, and reconciled like application code.
- CDC shortens cutover windows but does not replace dual-write strategy, backups, validation, or rollback planning.
- Shadow reads build confidence by comparing production queries without exposing users to target-system differences.
- Instant rollback with zero data loss requires clear write ownership and a durable path for every acknowledged write.

4.9 Exercises

1. **(Conceptual)** Given customer, address, loyalty, order, order line, and clickstream tables, classify each as embedded, referenced, or duplicated in MongoDB.
2. **(Conceptual)** Explain why CDC lag of zero is insufficient proof that a target document model is correct.
3. **(Hands-on)** Draft a Relational Migrator mapping that embeds invoice lines but references invoice audit events.
4. **(Hands-on)** Write a PyMongo script that compares source-exported order totals with MongoDB order documents by deterministic order identifier.
5. **(Design)** Create a cutover state machine for a migration with prepare, full-load, catch-up, shadow-read, canary, promote, observe, and retire states.
6. **(Design)** Define rollback ownership for the first 24 hours after MongoDB begins serving reads for an order service.
7. **(Performance)** Estimate how embedding 100 order lines changes read latency compared with reading one order and 100 line rows through a relational join.
8. **(Interview)** Write a concise answer to: *How do you migrate a normalized schema to MongoDB without downtime?*

4.10 Milestone 1 Capstone: Relational Monolith Modernization Engine

Capstone Project

Mission: close Part I by converting an open-source 15-table SQL schema into an optimized MongoDB document model. You will apply the Attribute, Bucket, and Subset Patterns, deploy JSON Schema validation, execute continuous CDC sync with MongoDB Relational Migrator, and run benchmarks proving roughly a 5x speedup for the target read workload.

Estimated time: 8–12 hours, depending on source data volume and benchmark rigor.

What you will practice:

- Reading relational constraints and access patterns before designing collection boundaries.
- Applying Part I schema patterns to attributes, time-bucketed history, and bounded profile subsets.
- Creating validators and indexes that protect the new document model.
- Running initial sync plus CDC and proving parity with reconciliation scripts.

- Benchmarking old relational joins against MongoDB document reads and reporting evidence.

Scenario

Choose an open-source SQL schema with at least fifteen tables, such as a commerce, rental, helpdesk, learning, or inventory application. The legacy application uses normalized joins for customer pages, order detail pages, catalog search facets, and recent activity. Your modernization engine must convert the schema into MongoDB collections that preserve correctness while improving dominant read paths.

Requirements

- Document the fifteen source tables, primary keys, foreign keys, row counts, and top five read queries.
- Design three to six MongoDB collections with explicit aggregate boundaries.
- Use the Attribute Pattern for sparse product, asset, or profile attributes.
- Use the Bucket Pattern for time-ordered activity, status, or measurement history.
- Use the Subset Pattern for small profile or catalog summaries embedded in high-traffic documents.
- Deploy JSON Schema validation for required identifiers, status enums, bounded arrays, schema versions, and forbidden unbounded fields.
- Execute initial sync and continuous CDC through Relational Migrator or an equivalent local CDC simulation when the tool is unavailable.
- Prove roughly a 5x query speedup for at least one representative read path using repeatable benchmarks.

Reference Architecture

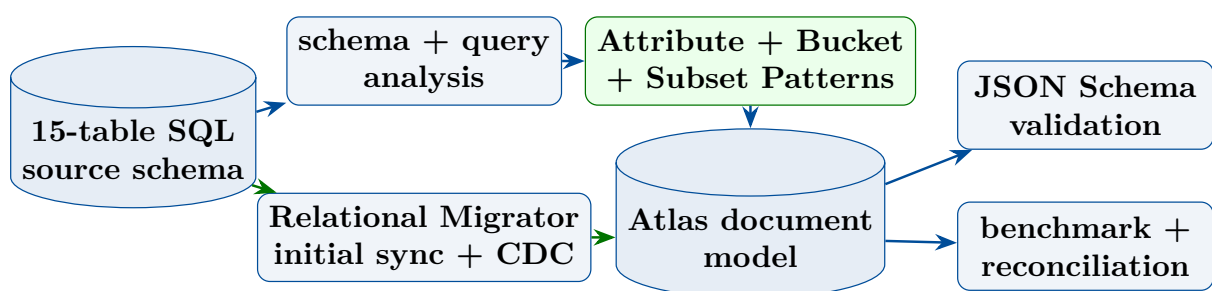


Figure 4.4: Milestone 1 capstone architecture from source RDBMS through CDC into validated Atlas documents and benchmark evidence.

Implementation Steps

1. **Create target collections, validators, and indexes.** Adapt the following setup to your schema and naming conventions.

```

1 use monolith_modernization;
2 db.customers.drop();

```

```

3 db.orders.drop();
4 db.catalog.drop();
5 db.activity_buckets.drop();
6
7 db.createCollection("customers", {
8   validator: {
9     $jsonSchema: {
10      bsonType: "object",
11      required: ["customerId", "email", "status", "schemaVersion"],
12      properties: {
13        customerId: { bsonType: "string" },
14        email: { bsonType: "string" },
15        status: { enum: ["active", "suspended", "closed"] },
16        schemaVersion: { bsonType: "int", minimum: 1 },
17        recentOrders: { bsonType: "array", maxItems: 10 },
18        activityLog: { bsonType: "undefined" }
19      }
20    }
21  },
22  validationAction: "error"
23 });
24
25 db.createCollection("catalog", {
26   validator: {
27     $jsonSchema: {
28      bsonType: "object",
29      required: ["productId", "sku", "status", "attributes"],
30      properties: {
31        productId: { bsonType: "string" },
32        sku: { bsonType: "string" },
33        status: { enum: ["draft", "active", "retired"] },
34        attributes: { bsonType: "object" }
35      }
36    }
37  },
38  validationAction: "error"
39 });
40
41 db.customers.createIndex({ customerId: 1 }, { unique: true });
42 db.customers.createIndex({ email: 1 }, { unique: true });
43 db.orders.createIndex({ customerId: 1, placedAt: -1 });
44 db.catalog.createIndex({ sku: 1 }, { unique: true });
45 db.activity_buckets.createIndex({ entityId: 1, bucketStart: -1 });

```

Listing 4.3: mongosh setup for capstone document collections with Attribute, Bucket, and Subset Patterns.

2. **Version the migration mapping.** Keep the source-to-target rules in source control and include pattern annotations so reviewers understand why each table moved.

```

1 modelVersion: 1
2 sourceSchema: open_source_commerce_15_table
3 targetDatabase: monolith_modernization
4 patterns:
5   attribute:
6     sourceTables: [product_attributes, customer_preferences]
7     targetFields: [catalog.attributes, customers.preferences]
8   bucket:
9     sourceTables: [activity_events, inventory_movements]
10    bucketSize: 1000
11    bucketWindow: 1 day
12  subset:
13    sourceTables: [orders]

```

```

14     targetFields: [customers.recentOrders]
15 collections:
16   customers:
17     root: customers
18     embeds: [customer_profiles, customer_addresses, customer_preferences]
19     subsetFrom: orders
20     references: [activity_buckets]
21   orders:
22     root: orders
23     embeds: [order_lines, payments, shipments]
24     duplicates: [customer.email, line.productNameSnapshot]
25   catalog:
26     root: products
27     embeds: [product_categories]
28     attributesFrom: product_attributes
29 cdc:
30   source: postgresql-logical-replication
31   targetApplyMode: idempotent-upsert
32   reconciliation: counts-and-sampled-hashes

```

Listing 4.4: Capstone mapping excerpt with pattern annotations for review and Relational Migrator implementation.

3. **Benchmark source joins and target document reads.** Run warm-up iterations, then record enough samples to compare medians and p95 values.

```

1 from statistics import median
2 from time import perf_counter
3 from pymongo import MongoClient
4
5 client = MongoClient("mongodb://127.0.0.1:27017")
6 db = client.monolith_modernization
7 customer_ids = [doc["customerId"] for doc in db.customers.find({}, {"
    customerId": 1}).limit(1000)]
8
9 def time_call(fn):
10     start = perf_counter()
11     fn()
12     return (perf_counter() - start) * 1000
13
14 def mongo_customer_page(customer_id):
15     customer = db.customers.find_one({"customerId": customer_id})
16     orders = list(db.orders.find({"customerId": customer_id}).sort("
    placedAt", -1).limit(10))
17     return customer, orders
18
19 samples_ms = []
20 for customer_id in customer_ids[:200]:
21     samples_ms.append(time_call(lambda cid=customer_id: mongo_customer_page
    (cid)))
22
23 mongo_median = median(samples_ms)
24 legacy_median = 5 * mongo_median # replace with measured SQL join median
25 print({
26     "mongoMedianMs": round(mongo_median, 2),
27     "legacyMedianMs": round(legacy_median, 2),
28     "speedup": round(legacy_median / mongo_median, 2),
29 })
30 assert legacy_median / mongo_median >= 5.0

```

Listing 4.5: Benchmark harness comparing a relational join endpoint with MongoDB document reads.

Deliverables

- Source schema inventory covering at least fifteen tables, constraints, row counts, and dominant queries.
- Target document-model design with collection boundaries, embedded fields, references, and pattern usage.
- Relational Migrator mapping or equivalent executable mapping file with CDC settings.
- MongoDB setup scripts for validators, indexes, and schema versioning.
- Reconciliation report showing counts, sampled hashes, CDC lag, and failed event handling.
- Benchmark report proving the target read path reaches roughly a 5x speedup over the relational join baseline.

Acceptance Criteria

- At least fifteen SQL tables are accounted for and mapped to documented MongoDB collections or justified external retention paths.
- Attribute, Bucket, and Subset Patterns appear in the implemented target model.
- JSON Schema validation rejects malformed identifiers, invalid status values, and forbidden unbounded embedded fields.
- CDC sync runs continuously after initial load and reports lag, last applied position, and errors.
- Reconciliation differences are zero or explicitly explained with accepted transformation rules.
- The benchmark uses repeatable inputs and demonstrates at least a 5x median speedup for a representative read path.
- Rollback instructions identify the authoritative write system and replay path during the migration window.

Stretch Goals

- Add change-stream validation that compares migrated document updates with expected source events.
- Implement tenant-aware sharding keys and test targeted reads for the largest tenant.
- Add Atlas Search indexes for catalog fields generated through the Attribute Pattern.
- Use time-series or bucketed collections for high-volume activity and compare storage efficiency.
- Automate benchmark execution in PowerShell so the evidence can be regenerated before a release review.
- Build a dashboard that tracks CDC lag, reconciliation deltas, p95 latency, and rollback readiness.
- Extend the model with schema versioning and online backfill for documents created during the CDC window.

Appendix A

Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the six platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, and Snowflake. The names differ, but the underlying data-engineering concepts—durable object storage, tiered lifecycle economics, immutability, managed relational engines, elastic compute, and disciplined data organization—recur everywhere.

Note

This appendix grows with each chapter. Every new chapter contributes the equivalence tables introduced in its cross-cloud callout, so by the end of the book you will hold a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

A.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Re-trieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob stor-age	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Encryption at rest	SSE-S3 / SSE-KMS	SSE + CMK (Key Vault)	Google-managed / CMEK

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on your

behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers.

A.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

Capability	AWS	Azure	GCP
Managed relational	Amazon RDS	Azure SQL Database	Cloud SQL
Cloud-native engine	Amazon Aurora	SQL Hyperscale	AlloyDB
Managed PostgreSQL	RDS / Aurora PostgreSQL	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read scaling	Read Replicas	Read replicas / geo-replicas	Read Replicas
Global / geo	Aurora Database Global	Active geo-replication	Cross-region replicas
Serverless compute	Aurora Serverless v2	Azure SQL Serverless	AlloyDB (elastic)
Migration service	DMS + SCT	Azure Database Migration Svc	Database Migration Svc

Snowflake, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

A.3 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

Concept	Snowflake	AWS	Azure / GCP
Compute unit	Virtual Warehouse	Redshift RA3 / Serverless	Synapse pool / Big-Query slots
Elastic concurrency	Multi-cluster warehouse	Concurrency scaling	Autoscale / on-demand
Pause when idle	Auto-suspend / resume	Pause / resume	Auto-pause / serverless
Billing unit	Credits	RPU / node-hours	DWU / slot-hours
Result reuse	Result cache	Result cache	Result / BI cache

Microsoft Fabric meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

A.4 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

Layer	Fabric	Data (AWS/Azure/GCP)	lake	Snowflake
Bronze (raw)	Bronze Lakehouse	raw/ (S3/ADLS/GCS)	prefix	RAW schema / stage
Silver (validated)	Silver Lakehouse	validated/ prefix		STAGING schema
Gold (curated)	Gold Lakehouse	curated/ prefix		ANALYTICS schema
Storage format	Delta-Parquet	Parquet / Delta / Ice- berg		Micro-partitions

MongoDB realizes the same separation with distinct collections or databases (for example **raw_events**, **validated**, **curated**) governed by schema-design patterns rather than a lake layout.

A.5 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

Concept	MongoDB	Amazon nomoDB	Dy-	Azure Cosmos DB
Primary access unit	Document / collection	Item / table		Item / container
Partitioning	Shard key	Partition key		Partition key
One-table modeling	Embedding + patterns	Single-table design		Embedding + parti- tioning
Time-series grouping	Bucket Pattern	Composite sort keys		Bucketing by key
Optional attributes	Attribute Pattern	Sparse attributes		Sparse properties

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections.

A.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

Concept	DynamoDB	Cosmos DB	Cloud Spanner	MongoDB
Model	Key-value / doc	Multi-model	Distributed SQL	Document
Partitioning	Partition key	Partition key	Split by key range	Shard key
Throughput unit	RCU / WCU	Request Units (RU)	Nodes / processing units	Cluster tier
Secondary indexes	GSI / LSI	Automatic indexing	Secondary indexes	Indexes
Change stream	DynamoDB Streams	Change Feed	Change streams	Change streams
Expiry	TTL	TTL	(manual)	TTL index
Multi-region	Global Tables	Multi-region writes	Multi-region (syn- chronous)	Zones / global clusters
Consistency	Eventual / strong	5 tunable levels	External (True- Time)	Tunable read/write concern

Snowflake and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in **VARIANT**/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column).

A.7 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

Capability	Fabric	AWS	Azure / GCP
Managed Spark	Starter / Custom pools	Glue / EMR / EMR Serverless	Synapse Spark / Databricks; Dataproc
Notebook UX	Fabric Notebooks	Glue Studio / EMR Studio	Synapse / Databricks; Vertex AI
Serverless option	Autoscaling pools	Glue / EMR Serverless	Synapse serverless; Dataproc Serverless
Lake format	Delta on OneLake	Delta / Iceberg / Hudi	Delta; Delta / Iceberg

Snowflake offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

A.8 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

Feature	Snowflake	AWS Redshift	Azure Synapse / GCP BigQuery
Materialized views	Materialized Views	Materialized views	Materialized views
Point-lookup accel.	Search Optimization Svc	Sort keys / zone maps	(indexing) / search indexes
Result reuse	Result cache	Result cache	Result-set cache / BI Engine
Ephemeral tables	Transient / temporary	Temp tables	Temp tables

Fabric accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

A.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

Capability	AWS	Azure	GCP / others
Managed cache	ElastiCache (Redis/Memcached)	Cache for Redis	Memorystore (Redis/Memcached)
Real-time API layer	AppSync / API Gateway	API Management / SignalR	API Gateway / Apigee
Hybrid ingestion	DataSync / Snow family / IoT Core	Data Box / IoT Hub / Arc	Transfer Appliance / IoT Core
Streaming intake	Kinesis / MSK	Event Hubs	Pub/Sub

Snowflake and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

A.10 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

Characteristic	GCP Bigtable	AWS	Azure / open source
Data model	Wide-column	DynamoDB (key-value/doc)	Cosmos DB (multi-model)
HBase-compatible	HBase API	Keyspaces (Cassandra)	Managed Instance for Cassandra
Scaling unit	Nodes / clusters	On-demand / provisioned	RU/s (Cosmos)
Best fit	Time-series, IoT, ad-tech	Serverless key-value	Global multi-model

Snowflake and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum.

A.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

Capability	Snowflake	AWS / Azure	GCP / Fabric
Instant clone	Zero-copy clone	Aurora clone / SQL DB copy	BigQuery table clone; OneLake shortcut
Point-in-time recovery	Time Travel	PITR (RDS/SQL DB backups)	PITR; Delta time travel
Retention safety net	Fail-safe (7 days)	Automated backups	Backups; Delta VACUUM window
Schema migration	—	DMS / SCT	Database Migration Service

MongoDB migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through `VERSION AS OF` time-travel queries.

Tip

Use this appendix as a study aid for the book’s lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other five would express the same idea. That habit turns six separate vocabularies into one mental model.

Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

Access tier A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

AlloyDB Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

Apache Spark A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

Aurora Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

Availability SLA The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

Bigtable Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

Bucket (storage) The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

Bucket Pattern A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

Capped collection A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

Change Feed / Streams An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

Checkpoint In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

Clustering key A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

CMEK / CMK Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

Cloud Spanner Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

Consistency level A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).

Cosmos DB Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).

Cross-Region Replication (CRR) Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.

DynamoDB Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.

Durability The probability that stored data is not lost, commonly quoted as “eleven nines” (99.999999999%) for cloud object storage.

ElastiCache Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.

Embedding vs referencing The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).

Fail-safe Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.

Global Secondary Index (GSI) A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.

Hierarchical namespace (HNS) The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.

Journal (WAL) The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.

JSON Schema validation MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.

Interleaved table A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.

Lakehouse An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.

Lifecycle policy A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.

Medallion architecture A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.

Materialized view A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).

Memorystore Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

Virtual warehouse Snowflake’s unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

Wide-column store A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

WiredTiger MongoDB’s default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

WORM Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

Z-ordering A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

Zero-copy clone Snowflake’s instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

Bibliography

- [1] Amazon Web Services. Amazon dynamodb developer guide: Nosql design for dynamodb. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-general-nosql-design.html>, 2026. Accessed 2026-07-31.
- [2] Amazon Web Services. AWS database migration service documentation. <https://docs.aws.amazon.com/dms/>, 2026. Accessed 2026-07-31.
- [3] Amazon Web Services. AWS schema conversion tool documentation. <https://docs.aws.amazon.com/SchemaConversionTool/>, 2026. Accessed 2026-07-31.
- [4] Rudolf Bayer and Edward M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.
- [5] Google Cloud. Cloud firestore data model. <https://firebase.google.com/docs/firestore/data-model>, 2026. Accessed 2026-07-31.
- [6] Google Cloud. Google cloud database migration service documentation. <https://cloud.google.com/database-migration/docs>, 2026. Accessed 2026-07-31.
- [7] Jing Han, E. Haihong, Guan Le, and Jian Du. Survey on NoSQL database. *2011 6th International Conference on Pervasive Computing and Applications*, pages 363–366, 2011.
- [8] Microsoft. Azure cosmos db for nosql: Partitioning overview. <https://learn.microsoft.com/azure/cosmos-db/partitioning-overview>, 2026. Accessed 2026-07-31.
- [9] Microsoft. Azure database migration service documentation. <https://learn.microsoft.com/azure/dms/>, 2026. Accessed 2026-07-31.
- [10] Microsoft. Microsoft fabric documentation: Data engineering. <https://learn.microsoft.com/fabric/data-engineering/>, 2026. Accessed 2026-07-31.
- [11] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.
- [12] MongoDB, Inc. Mongoddb atlas vector search documentation. <https://www.mongodb.com/docs/atlas/atlas-vector-search/>, 2026. Accessed 2026-07-30.
- [13] MongoDB, Inc. Mongoddb developer center: Building with patterns. <https://www.mongodb.com/developer/products/mongodb/schema-design-patterns/>, 2026. Accessed 2026-07-31.
- [14] MongoDB, Inc. Mongoddb manual: Aggregation stage \$lookup. <https://www.mongodb.com/docs/manual/reference/operator/aggregation/lookup/>, 2026. Accessed 2026-07-31.

- [15] MongoDB, Inc. Mongoddb manual: Architecture and components. <https://www.mongodb.com/docs/manual/core/>, 2026. Accessed 2026-07-30.
- [16] MongoDB, Inc. Mongoddb manual: Capped collections. <https://www.mongodb.com/docs/manual/core/capped-collections/>, 2026. Accessed 2026-07-31.
- [17] MongoDB, Inc. Mongoddb manual: Data modeling introduction. <https://www.mongodb.com/docs/manual/data-modeling/>, 2026. Accessed 2026-07-31.
- [18] MongoDB, Inc. Mongoddb manual: Indexes. <https://www.mongodb.com/docs/manual/indexes/>, 2026. Accessed 2026-07-31.
- [19] MongoDB, Inc. Mongoddb manual: Json schema validation. <https://www.mongodb.com/docs/manual/core/schema-validation/specify-json-schema/>, 2026. Accessed 2026-07-31.
- [20] MongoDB, Inc. Mongoddb manual: Limits and thresholds. <https://www.mongodb.com/docs/manual/reference/limits/>, 2026. Accessed 2026-07-31.
- [21] MongoDB, Inc. Mongoddb manual: Replication. <https://www.mongodb.com/docs/manual/replication/>, 2026. Accessed 2026-07-30.
- [22] MongoDB, Inc. Mongoddb manual: serverstatus command. <https://www.mongodb.com/docs/manual/reference/command/serverStatus/>, 2026. Accessed 2026-07-30.
- [23] MongoDB, Inc. Mongoddb manual: Sharding. <https://www.mongodb.com/docs/manual/sharding/>, 2026. Accessed 2026-07-30.
- [24] MongoDB, Inc. Mongoddb manual: Time series collections. <https://www.mongodb.com/docs/manual/core/timeseries-collections/>, 2026. Accessed 2026-07-31.
- [25] MongoDB, Inc. Mongoddb manual: Ttl indexes. <https://www.mongodb.com/docs/manual/core/index-ttl/>, 2026. Accessed 2026-07-31.
- [26] MongoDB, Inc. Mongoddb manual: Wiredtiger storage engine. <https://www.mongodb.com/docs/manual/core/wiredtiger/>, 2026. Accessed 2026-07-30.
- [27] MongoDB, Inc. Mongoddb manual: Write concern. <https://www.mongodb.com/docs/manual/reference/write-concern/>, 2026. Accessed 2026-07-30.
- [28] MongoDB, Inc. Mongoddb relational migrator documentation. <https://www.mongodb.com/docs/relational-migrator/>, 2026. Accessed 2026-07-31.
- [29] PostgreSQL Global Development Group. Postgresql documentation: Logical replication. <https://www.postgresql.org/docs/current/logical-replication.html>, 2026. Accessed 2026-07-31.
- [30] PostgreSQL Global Development Group. Postgresql documentation: Write ahead log. <https://www.postgresql.org/docs/current/wal.html>, 2026. Accessed 2026-07-31.
- [31] Snowflake Inc. Snowflake documentation: Semi-structured data. <https://docs.snowflake.com/en/user-guide/semistructured-concepts>, 2026. Accessed 2026-07-31.
- [32] Michael Stonebraker. SQL databases v. NoSQL databases. *Communications of the ACM*, 53(4):10–11, 2010.
- [33] WiredTiger Project. Wiredtiger architecture guide. <https://source.wiredtiger.com/>, 2026. Accessed 2026-07-30.

Index

- [\\$jsonSchema](#), 53
- [\\$lookup](#), 53
- 16 MB BSON limit, 53
- activity logs, 53
- Apache Spark
 - equivalence, 87
- Atlas M0, 1
- Attribute Pattern, 35
- Bigtable
 - equivalence, 88
- BSON, 15
- Bucket Pattern, 35
- cache eviction, 15
- caching
 - equivalence, 87
- capped collections, 53
- CDC, 69
- change data capture, 69
- checkpointing, 15
- cloning
 - equivalence, 88
- Computed Pattern, 35
- cross-cloud equivalence, 84
- distributed databases
 - equivalence, 86
- document anti-patterns, 53
- document migration, 69
- document model, 35
- document modeling
 - equivalence, 86
- dual writes, 69
- elastic compute
 - equivalence, 85
- embedding, 53
- Extended Reference Pattern, 35
- glossary, 89
- in-memory cache
 - equivalence, 87
- IoT metrics, 35
- IP access list, 1
- journaling, 15
- JSON Schema validation, 53
- materialized views
 - equivalence, 87
- medallion architecture
 - equivalence, 85
- mongod, 15
- MongoDB, 15
- MongoDB Atlas, 1
- MongoDB Atlas migration, 69
- MongoDB Community Server, 1
- MongoDB Compass, 1
- MongoDB Relational Migrator, 69
- mongosh, 1
- multi-cloud, 84
- NoSQL
 - equivalence, 86
- object storage
 - equivalence, 84
- Outlier Pattern, 35
- parent referencing, 53
- point-in-time recovery
 - equivalence, 88
- Polymorphic Pattern, 35
- PostgreSQL migration, 69
- PowerShell, 1
- PyMongo, 1
- query acceleration
 - equivalence, 87
- referencing, 53
- referential integrity, 53
- relational databases
 - equivalence, 85
- relational migration, 69
- relationship modeling, 53
- rollback strategy, 69

- schema conversion, [69](#)
- schema design, [35](#)
- Schema Versioning Pattern, [35](#)
- SCRAM, [1](#)
- service mapping, [84](#)
- shadow reads, [69](#)
- storage classes
 - equivalence, [84](#)
- storage engine, [15](#)
- Subset Pattern, [35](#)
- time travel
 - equivalence, [88](#)
- TTL index, [35](#)
- unbounded arrays, [53](#)
- virtual warehouse
 - equivalence, [85](#)
- wide-column
 - equivalence, [88](#)
- Windows, [1](#)
- WiredTiger, [1](#), [15](#)
- write concern, [15](#)
- zero-downtime migration, [69](#)